

Computer Networks

Lecture 6

Shmulik Kaplan

Today

- Consensus Algorithm – Byzantine failures
- Application level – APIs
- Transport layer
  - Recap
  - Multiplexing & Demultiplexing
  - UDP
  - TCP Error control + *in-order* byte-stream data transfer
    - What can go wrong

Today

- Consensus Algorithm – Byzantine failures
- Application level – APIs
- Transport layer
  - Recap
  - Multiplexing & Demultiplexing
  - UDP
  - TCP Error control + *in-order* byte-stream data transfer
    - What can go wrong

Binary Consensus for Byzantine Failures where $n \geq 4t+1$

	Byzantine failures
number of processes	$n \geq 4t+1$
number of rounds	$2(t+1)$

The notes cover only the second algorithm.

Presentation of this algorithm is based on:

A Simple proof of a simple consensus algorithm
by J. Misra
Information Processing Letters, (1989) 33:1, 21-24.

Asymptotical optimal Distributed consensus
by P. Berman and J.A. Garay
Proc. ICALP 89. In: LNCS, 372, 80—94, 1989.

Binary Consensus for Byzantine Failures where n ≥ 4t+1

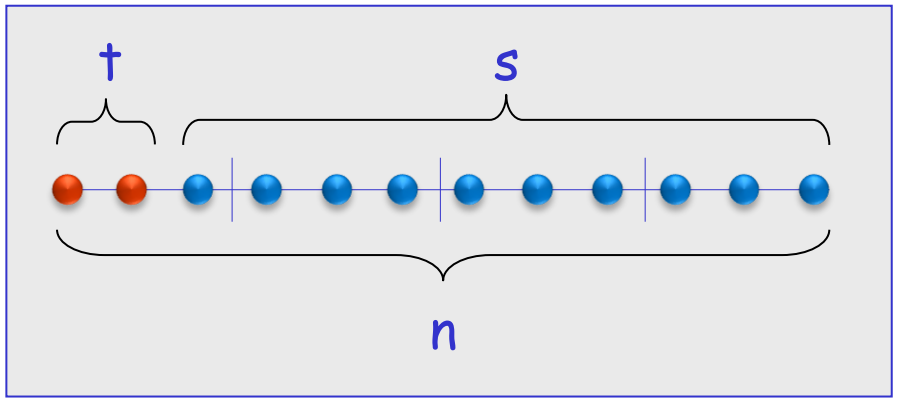
Model:

- Synchronous network
- # of processes n is a priori known (for simplicity assume n is odd)
- Unique identifiers: 1,…,n
- Input is \in {0,1}
- Network: every process is connected to every other process (clique)
- Communication: message passing
- At most t Byzantine failures, where n ≥ 4t+1
- # of failures t is a priori known
- No failures of links or messages
- Startup condition: all at the same time

Binary Consensus for Byzantine Failures where $n \geq 4t+1$

Notations:

- s – correct processes; $s > 3n/4$ (a priori known)
- t – faulty processes; $t < n/4$ (a priori known)
- $n = s + t$ (for simplicity assume that n is odd)



Binary consensus for Byzantine Failures where n ≥ 4t+1

Algorithm for process p_i with input in_i

Processes communicate in rounds. Each round k (k>o), has two sub-rounds.

The algorithm:

v = in_i

for k=1 to t+1 do

1. send v to all the processes, and receive messages from all the processes;
2. if my-id = k then v = "majority value from step 1"; send v to all other processes

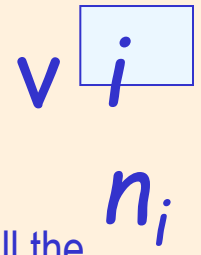
   else receive a value from process k;

if in step 1 you have received s or more messages with the same value (including your own) then assign this value to v

else assign to v the value you have received in step 2.

end loop

decide on v



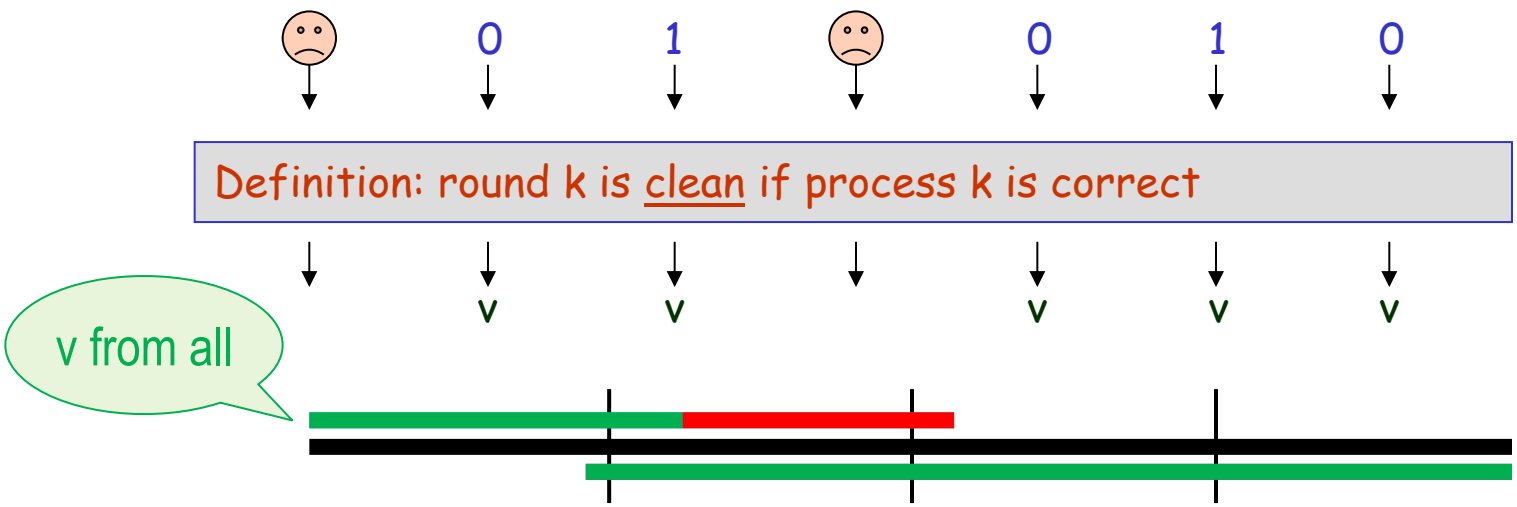
Binary Consensus for Byzantine Failures where n ≥ 4t+1



Binary Consensus for Byzantine Failures where n ≥ 4t+1

Notation: v is the majority value of process k in round k

Lemma 1: round k is clean ➔ same output v



➔ process k got v from > n/2 processes

➔ process k got v from > n/4 correct processes

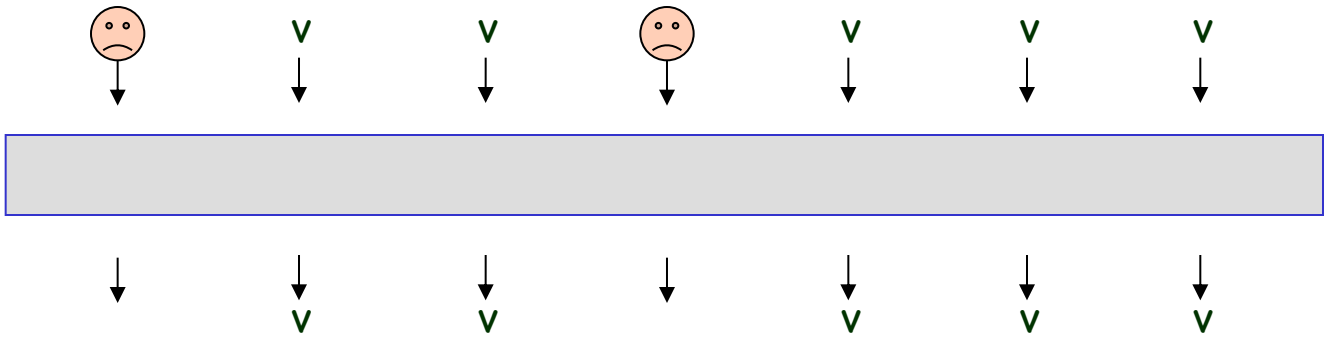
➔ every process got v from > n/4 correct processes

➔ if a process got at least s same messages ➔ must be v

➔ if a process got less than s same messages ➔ choose v

Binary Consensus for Byzantine Failures where n ≥ 4t+1

Lemma 2: same input (of all correct processes) ➔ same output



➔ # of correct processes, *s* > 3n/4

➔ each correct process gets *s* messages with the value v

➔ each correct process chooses the value v

time

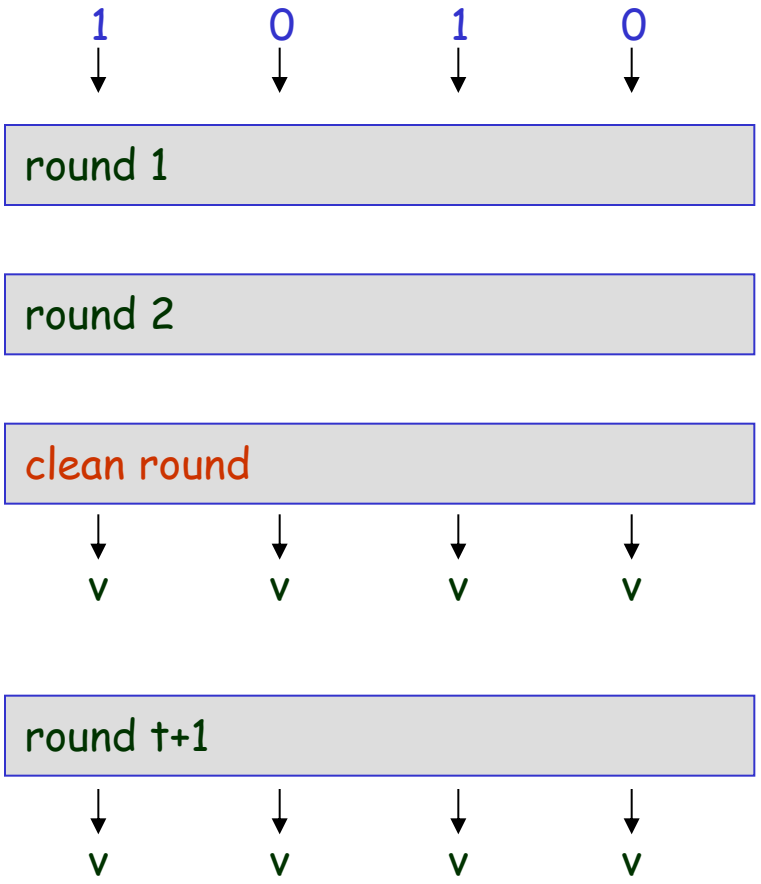
Binary Consensus for Byzantine Failures where n ≥ 4t+1

Lemma 1: clean round ➔ same output

Lemma 2: same input ➔ same output

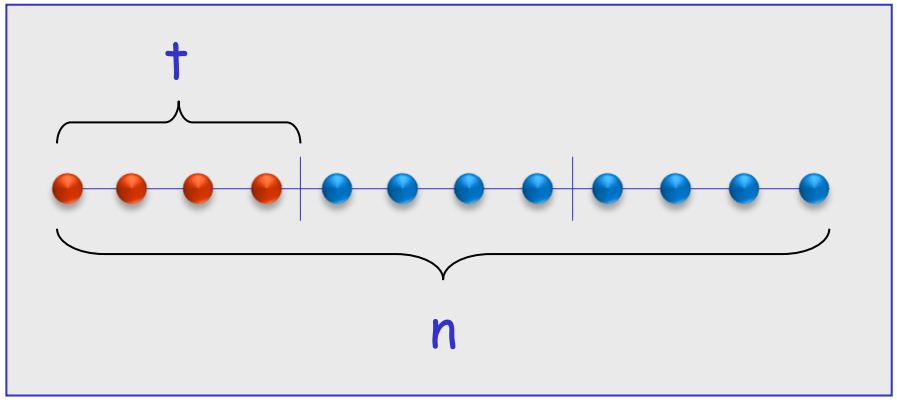
Thm 1: Agreement ✓

Thm 2: Validity ✓



Impossibility of Consensus with n/3 Faulty Byzantine Process

Theorem: There is no synchronous algorithm for n processes that solves the consensus problem using send/receive messages in the presence of n/3 (or more) faulty Byzantine process.



What's about real life?

- **No guarantees on timing… except eventually**
  The system may be asynchronous most of the time, but eventually message delays fall within bounds.
  (This is the "partial synchrony" model.)
- **Timeout-based failure detection**
  We treat slow nodes as failed because we cannot distinguish delay from crash.
- **Quorum-based progress**
  Synchronization advances when a majority of nodes respond within timing expectations.
- **Authenticity instead of trust**
  Nodes don't have to trust each other — but they must verify message origins (signatures, MACs, tokens).

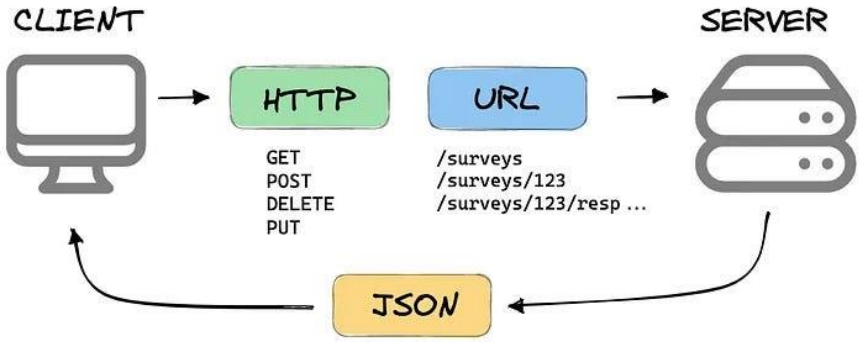
Why Byzantine?



Today

- ✔ Consensus Algorithm – Byzantine failures
- Application level – APIs
- Transport layer
  - Recap
  - Multiplexing & Demultiplexing
  - UDP
  - TCP Error control + *in-order* byte-stream data transfer
    - What can go wrong

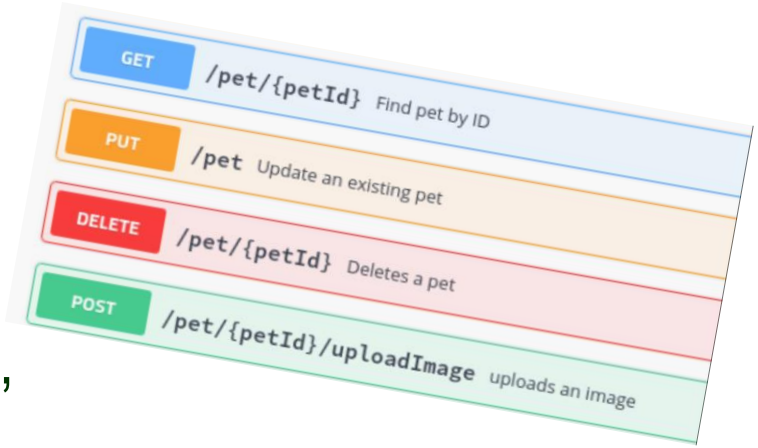
Let's communicate using an API



- **An API is a defined way for one software component to request capabilities from another.**
  It creates a predictable, shared contract for remote interaction.
- **API = Application Programming Interface**
- **What APIs are used for**
  - Accessing or retrieving information
  - Configuring or controlling remote systems
  - Creating, updating, or deleting data
  - Triggering operations or workflows
  - Querying status or health of a component
  - Integrating independent systems in a reliable way

HTTP and HTTPS for web based API

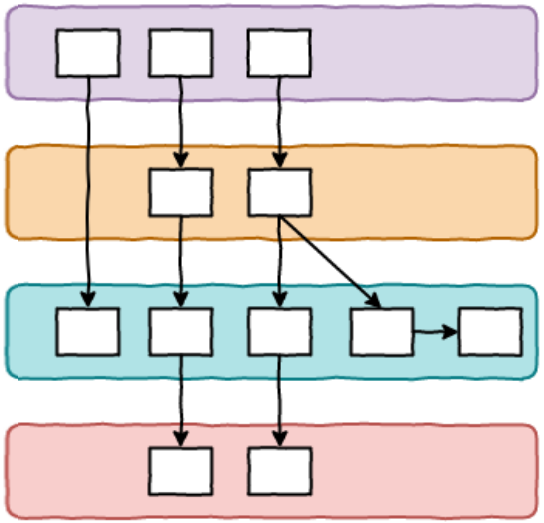
- **HTTP is the protocol that carries application-level messages between a client and a server.**
  It defines *how* a request is sent and *how* a response is returned, but it does not define the business meaning of the data.
  HTTPS adds encryption and authentication, but keeps the same request–response model.
- **Key concepts**
  - Standard request/response exchange
  - Structured message format: method, URI, headers, body
  - Stateless communication model
  - HTTPS adds TLS for confidentiality and integrity



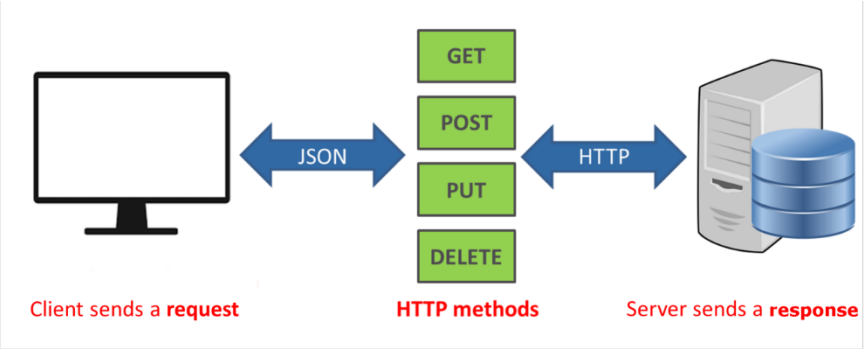
Web APIs on Top of HTTP

- **A Web API defines the meaning of the messages exchanged over HTTP.**
  It specifies what operations exist, what data structures are expected, and how clients should interpret each response. HTTP provides the envelope; the Web API provides the semantics.

- **What a Web API defines**
  - The available endpoints and their purposes
  - The structure and types of the data exchanged
  - Expected behaviors for each operation
  - Rules for success, failure, and error reporting



Why Not Just Use HTTP As It Is?



- **HTTP already gives us a simple, elegant interaction model.**
  It defines clear methods, predictable request–response flows, and a consistent message structure designed by Tim Berners-Lee for the open web.
  In principle, this should be enough for any API.
- **What HTTP already gives us**
  - A small, well-defined set of methods
    - Safe (Get, Options)
    - Idempotent (Put, Delete)
    - Neither (Post, Patch)
  - A clean message format (method, URI, headers, body)
  - Uniform request→response behavior
  - A universal standard understood everywhere

The Reality — Developers Misuse HTTP

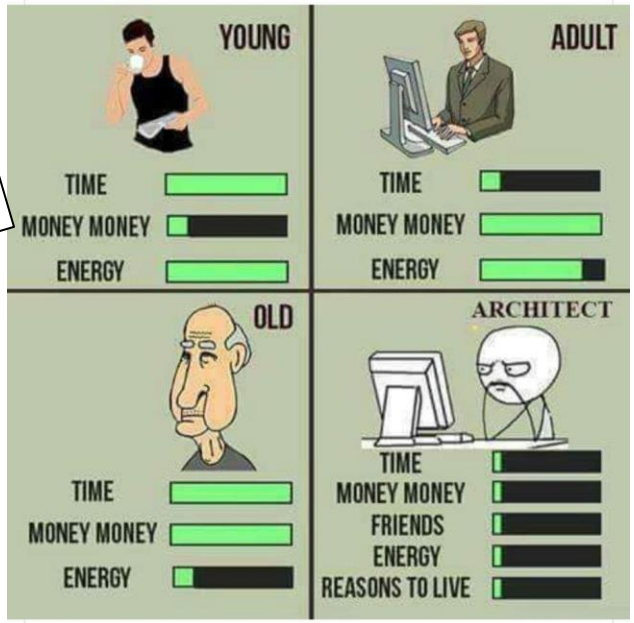
- **In practice, developers often ignore the intended semantics of HTTP.**

  They overload methods, skip conv… and treat the protocol as a generic tunnel.

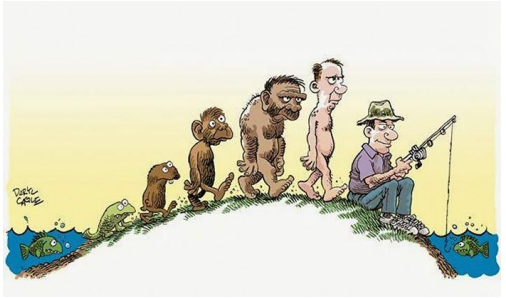
- **Commo…**
  - …semantics
  - …g operations in URLs instead of using methods
  - Mixing parameters, actions, resources unpredictably and Responses

Result:

We need **API architectural styles** restore structure, clarity, and consistency.



The Evolution of API Architectural Styles I



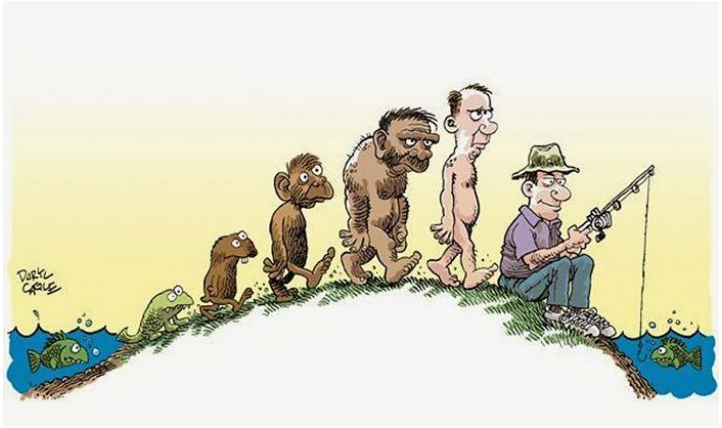
- **SOAP (Simple Object Access Protocol)**
  Built for strongly typed, enterprise-grade interfaces.
  - *Need:* Needed formal contracts, strict schemas, and tooling support.
  - *Problem:* SOAP was too heavy, rigid, and complex for the open web.

- **REST (REpresentational State Transfer)**
  Emphasized simplicity, resource orientation, and uniform HTTP semantics.
  - *Problem:* REST's resource model didn't fit action-oriented workloads or strongly typed systems.
  - *Problem:* REST endpoints often caused under-fetching and over-fetching for modern, data-heavy clients.

The Evolution of API Architectural Styles II

- **RPC-style APIs (Remote Procedure Call)**
  Including **JSON-RPC** and **gRPC (Google Remote Procedure Call)**.

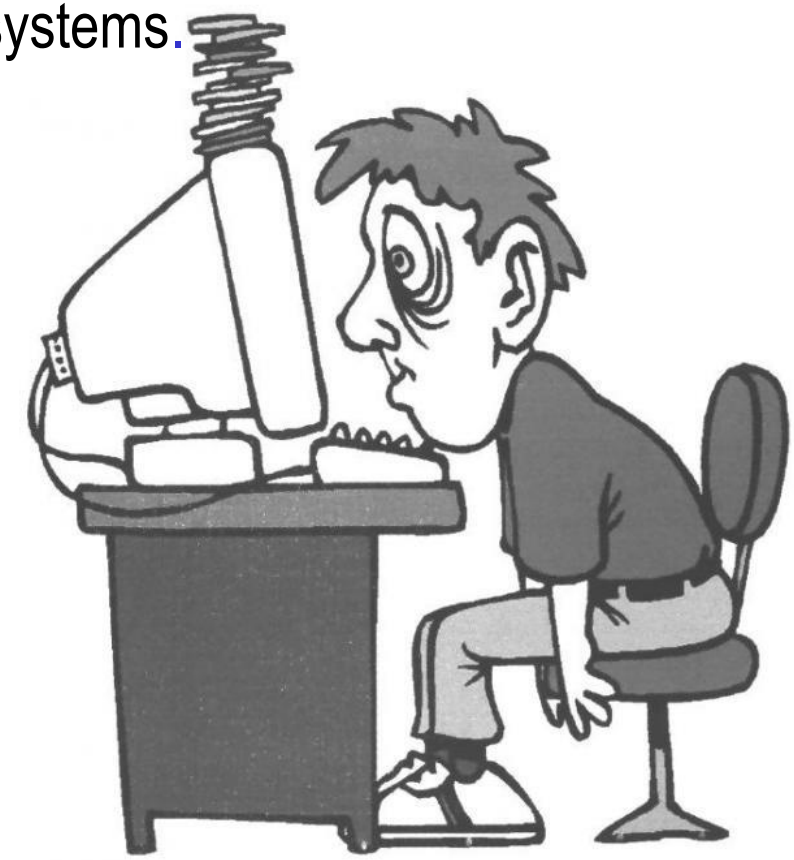
- **GraphQL (Graph Query Language)**
  A flexible query language allowing clients to request exactly the data they need.
  Problem: REST endpoints often caused under-fetching and over-fetching for modern, data-heavy clients.



REST — The Dominant Style for Web APIs

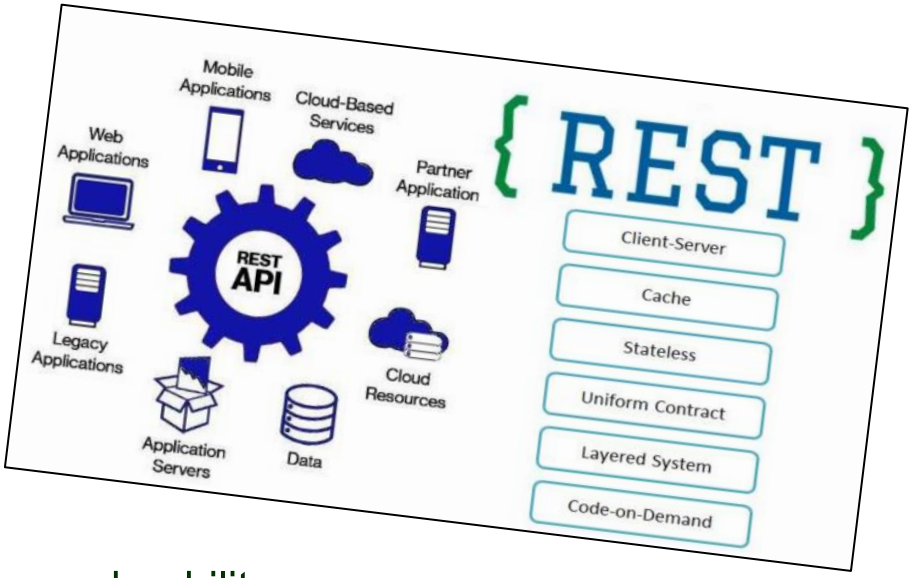
- **REST became the most widely adopted API style because it aligns naturally with how the web already works.**
  It uses simple, uniform operations on stable resource identifiers, and it scales well for large, distributed systems.

- **Core REST ideas**
  - Resources identified by URIs
  - HTTP methods express actions
  - Stateless interactions
  - Standardized representations (typically JSON)



REST API

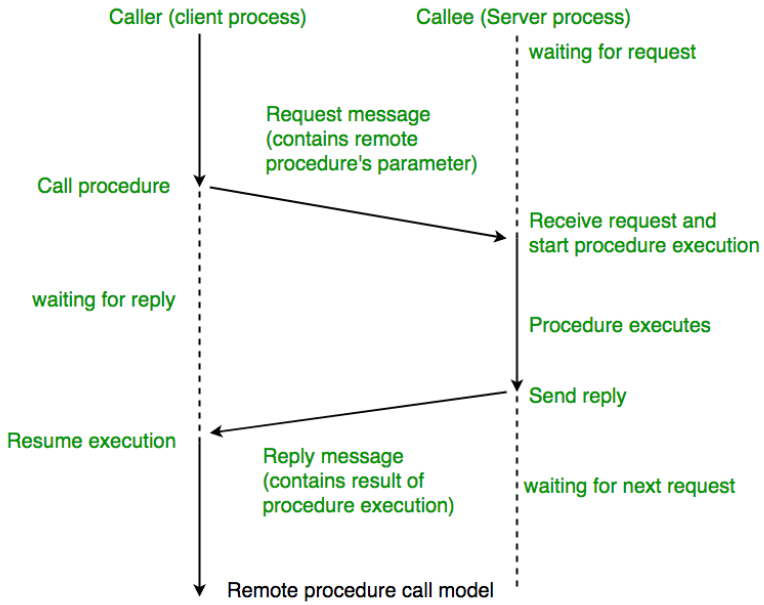
- The formal REST constraints
  - Client/Server – Clients are separated from servers by a well-defined interface
  - Stateless – A specific client does not consume server storage when the client is "at rest"



  - Cache – Responses indicate their own cacheability
  - Uniform interface
  - Layered system – A client cannot ordinarily tell whether it is connected directly to the end server, or to an intermediary along the way
  - Code on demand (optional) – Servers are able to temporarily extend or customize the functionality of a client by transferring logic to the client that can be executed within a standard virtual machine

RPC-Style APIs (Remote Procedure Call)

- **RPC-style APIs model the API as a set of remote function calls rather than resources.**
  They focus on invoking operations directly, passing parameters, and receiving results, similar to calling a local function but over a network.
- **Why RPC emerged**
  - Some systems are action-oriented, not resource-oriented
  - REST felt unnatural for operations like "startJob", "compileCode", "resizeImage"
  - Developers wanted stronger typing and clearer contracts
- **Examples**
  - JSON-RPC
  - XML-RPC
  - gRPC (Google Remote Procedure Call)

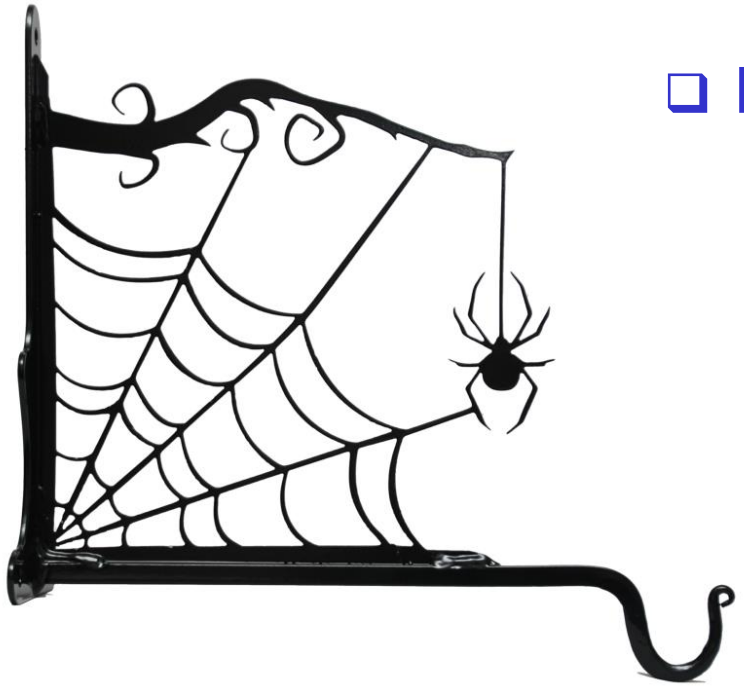


Server-Initiated Communication

- **Not all interactions fit the traditional request–response model.**
  Some applications require the server to push information to clients as events happen, without waiting for the client to ask.
- **Common scenarios**
  - **Real-time status updates**
    (e.g., job progress, system health changes, sensor events)
  - **Event notifications**
    (e.g., “payment completed”, “file uploaded”, “new message arrived”)
  - **Collaborative or interactive systems**
    (e.g., shared document editing, live dashboards, multiplayer systems)

Webhooks — Server-to-Client Events via HTTP

- **With Webhooks, the client exposes a URL, and the server sends an HTTP request to that URL when an event occurs.** This avoids polling and enables real-time event delivery using standard HTTP.



- **Key characteristics**
  - The **client provides a callback URL**
  - The **server sends an HTTP POST** when an event happens
  - Lightweight and easy to integrate
  - Ideal for event-driven workflows (payments, uploads, notifications)



WebSockets — Bidirectional Communication Over a Client-Initiated Connection

- **WebSockets create a persistent, full-duplex channel, but the client always initiates the connection.**
  The server never connects back to the client; it only sends data through the already-open socket the client created.
- **Key characteristics**
  - The **client opens** the WebSocket connection
  - The server **accepts**, then both sides can send messages
  - No inbound connections to the client (unlike Webhooks)
  - Ideal for real-time, low-latency communication (chat, dashboards, games)

Web API in real life

- Document
- Swagger

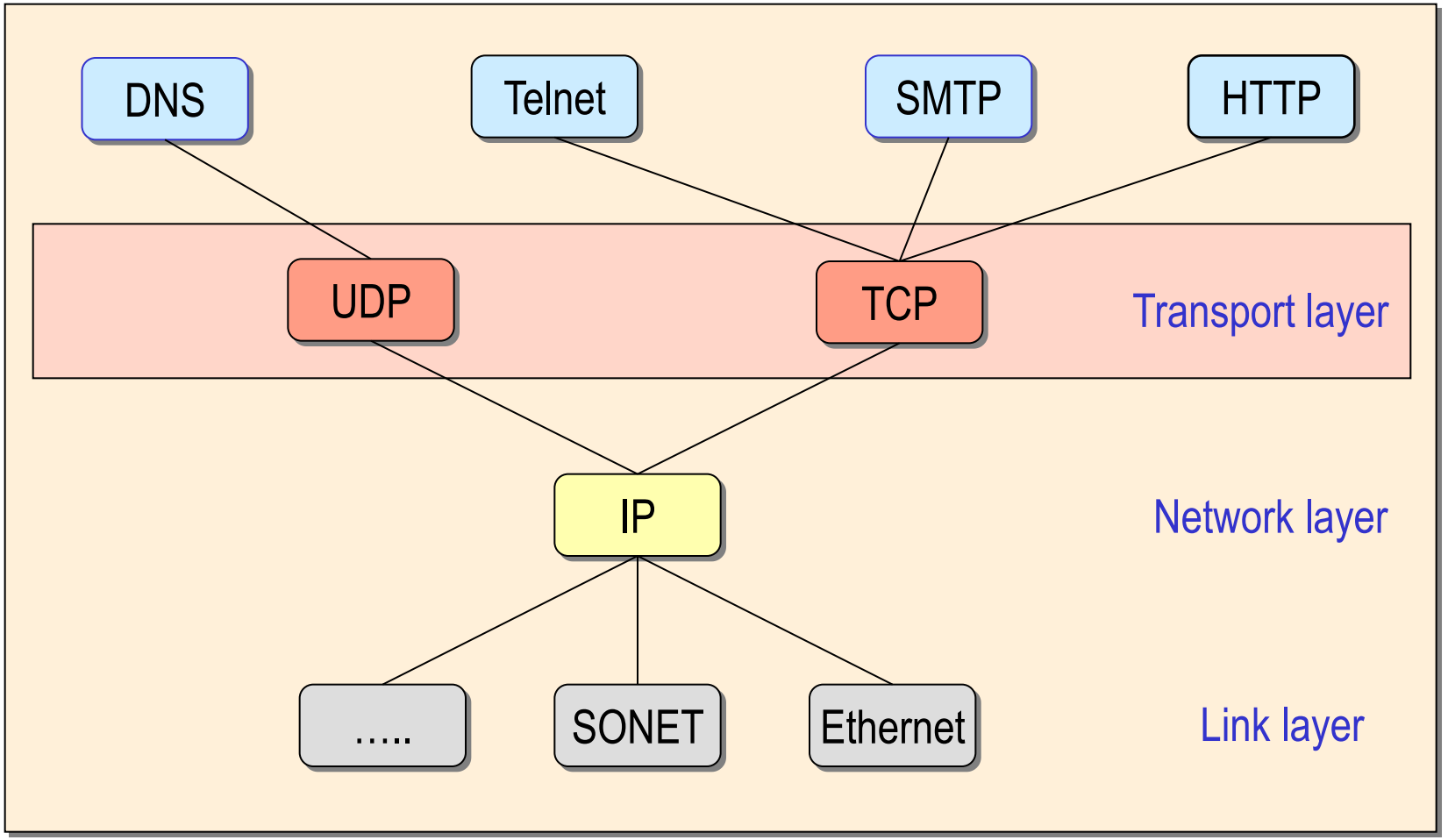


Today

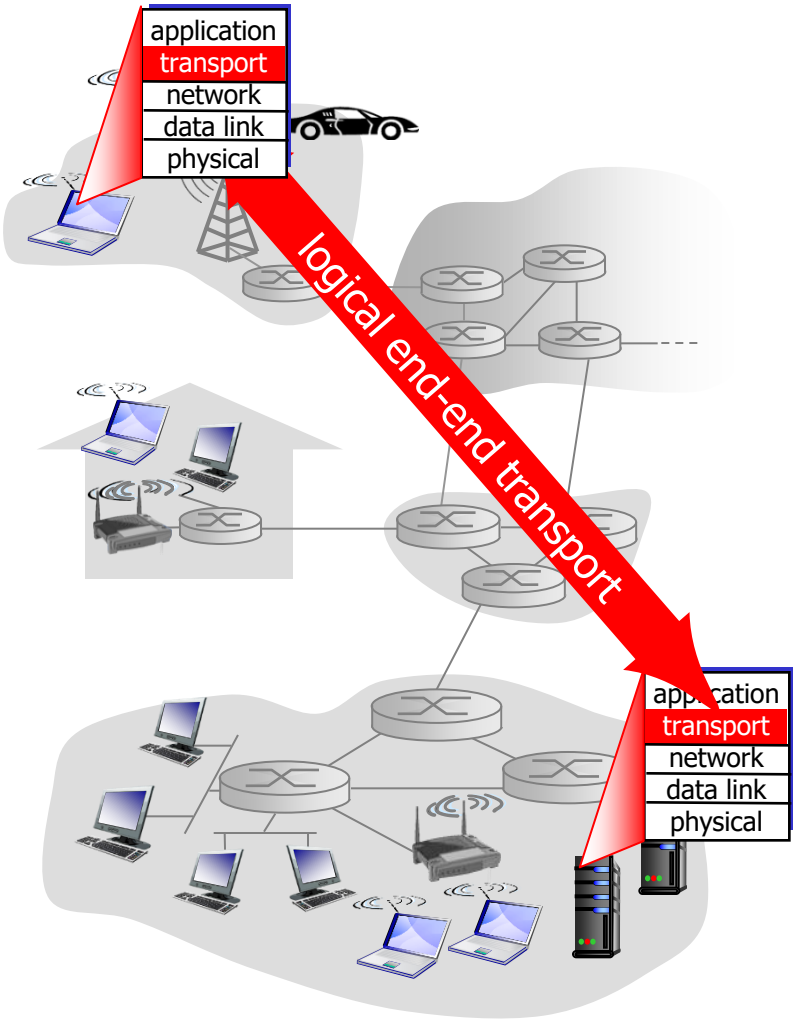
- ✔ Consensus Algorithm – Byzantine failures
- ✔ Application level – APIs
- Transport layer
  - Recap
  - Multiplexing & Demultiplexing
  - UDP
  - TCP Error control + *in-order* byte-stream data transfer
    - What can go wrong

Computer Networks

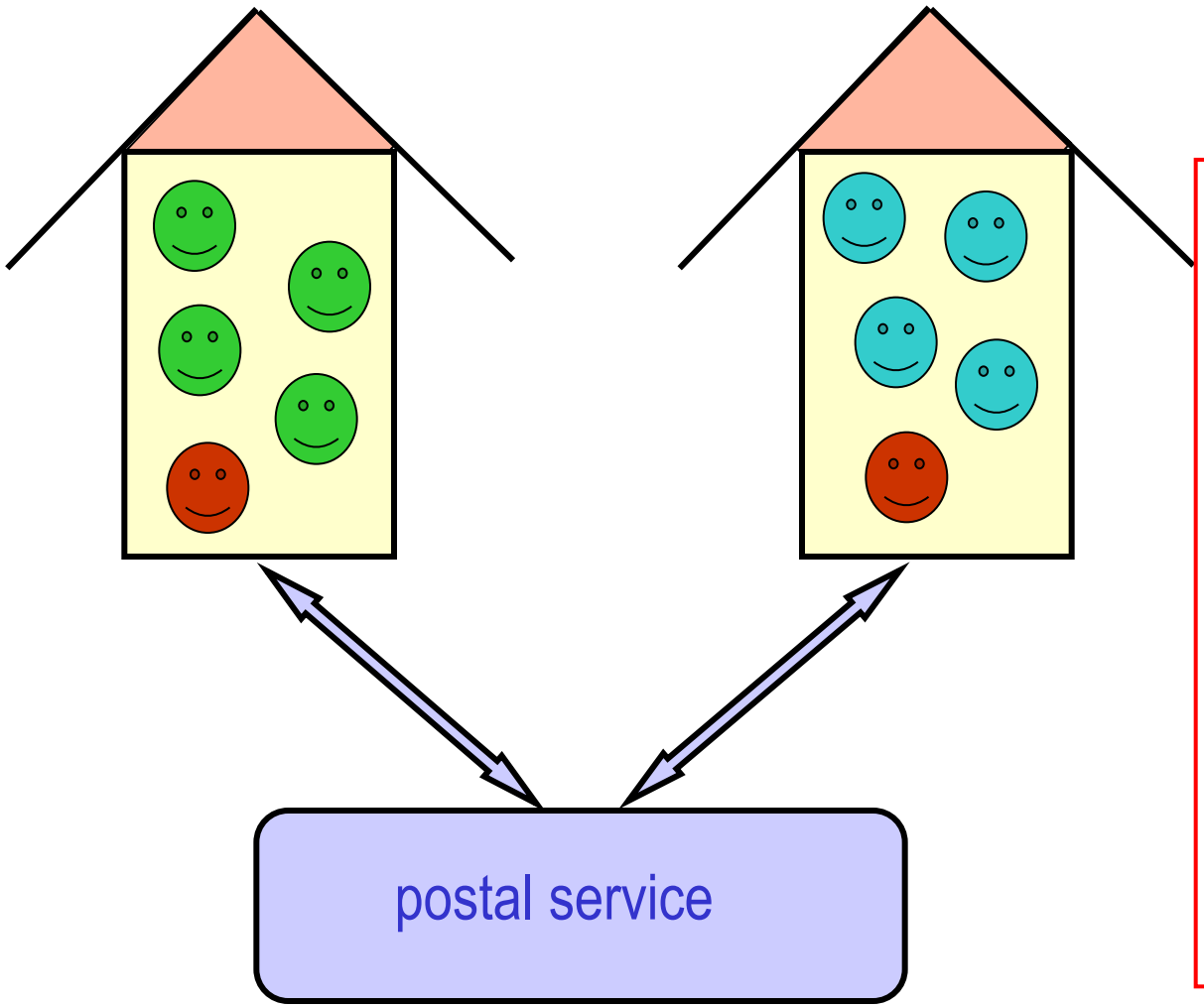
Gadi Taubenfeld



Transport Layer



An “old” example

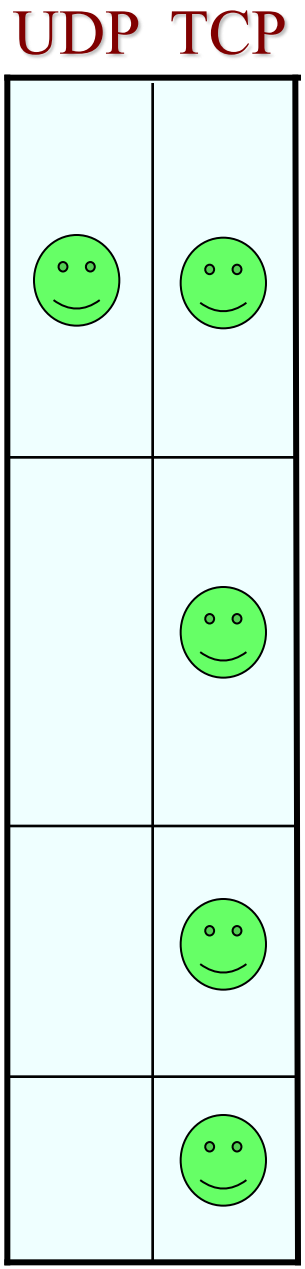


Household analogy:

- processes
  = the “blue” & “green” kids
- app messages
  = letters in envelopes
- hosts
  = houses
- transport protocol
  = the “red” kids
- network-layer protocol
  = postal service

Transport layer services

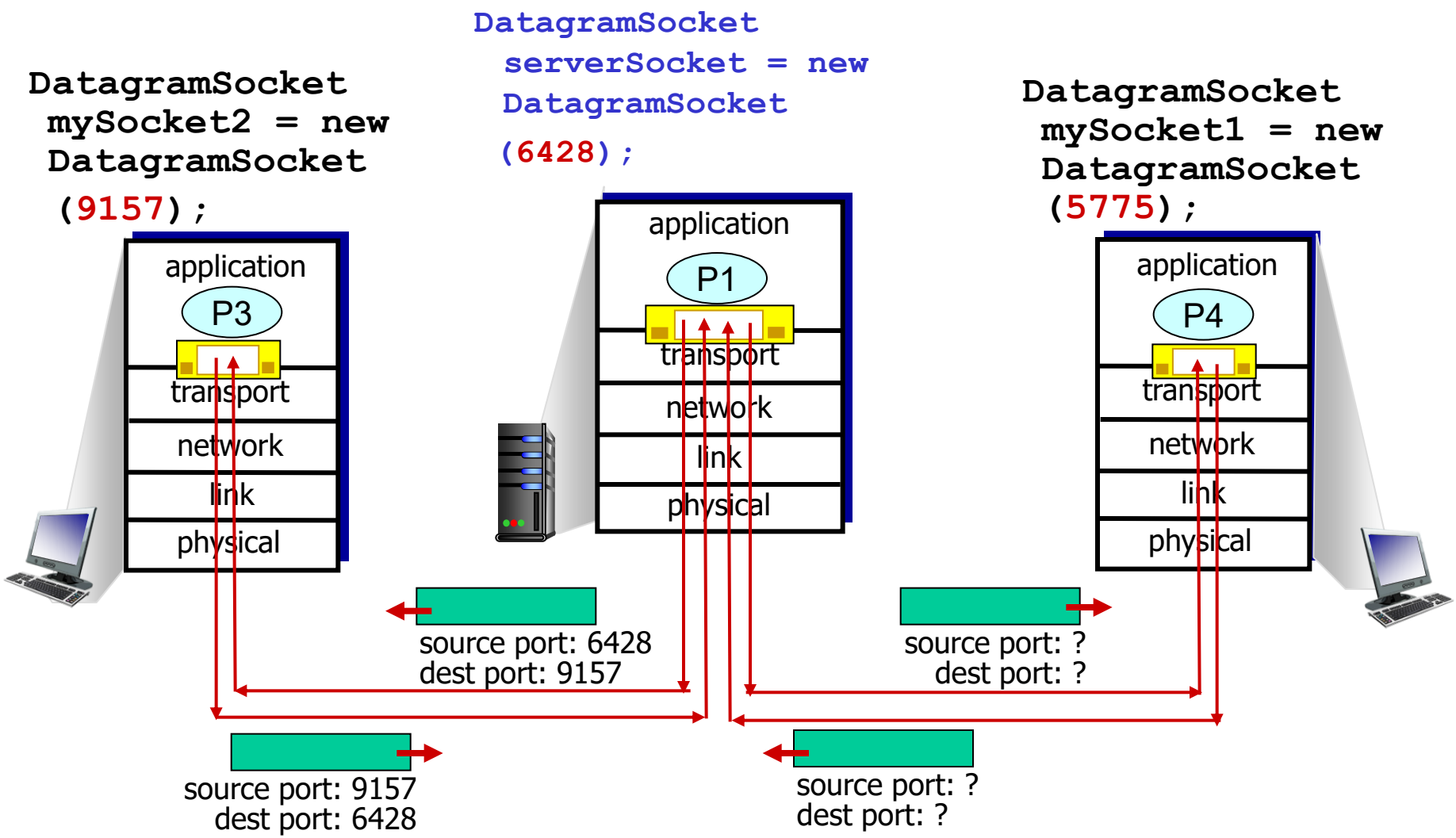
- Multiplexing/demultiplexing
  - adds an application-specific identifier (*port number)* so that receiving end-system can hand in incoming packet to the correct application
- Error control + *in-order* byte-stream data transfer
  - message will reach destination despite packet loss, corruption or duplication
  - retransmit lost packets; detect, discard, and retransmit corrupted packets; detect and discard duplicated packets
- Flow control
  - match transmission rate to rate currently sustainable on the path to destination, and at the destination itself
- Congestion control
  - senders “slow down sending rate” when network congested



Today

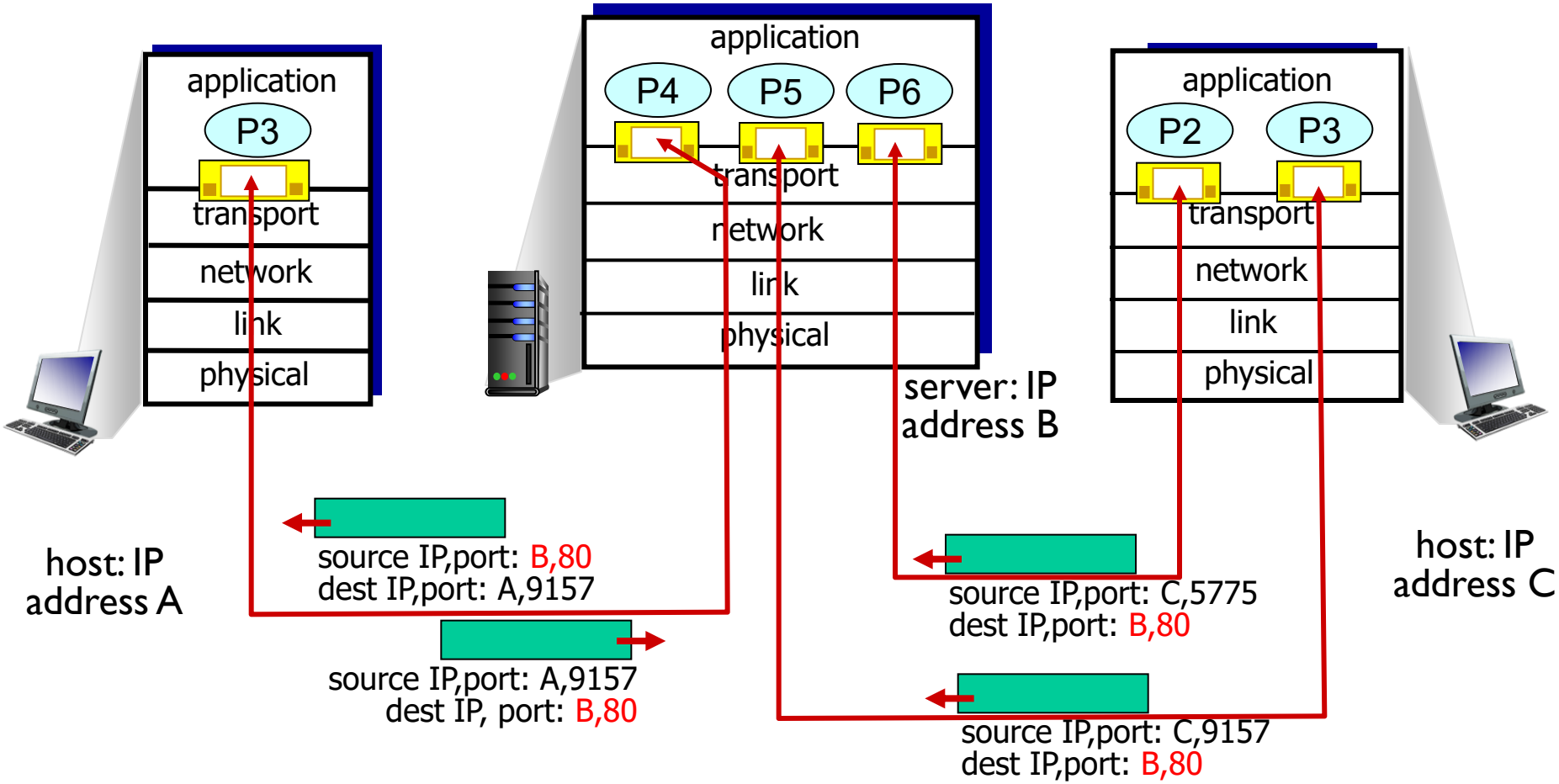
- ✓ Consensus Algorithm – Byzantine failures
- ✓ Application level – APIs
- Transport layer
  - ✓ Recap
  - Multiplexing & Demultiplexing
  - UDP
  - TCP Error control + *in-order* byte-stream data transfer
    - What can go wrong

Connectionless demultiplexing (UDP)

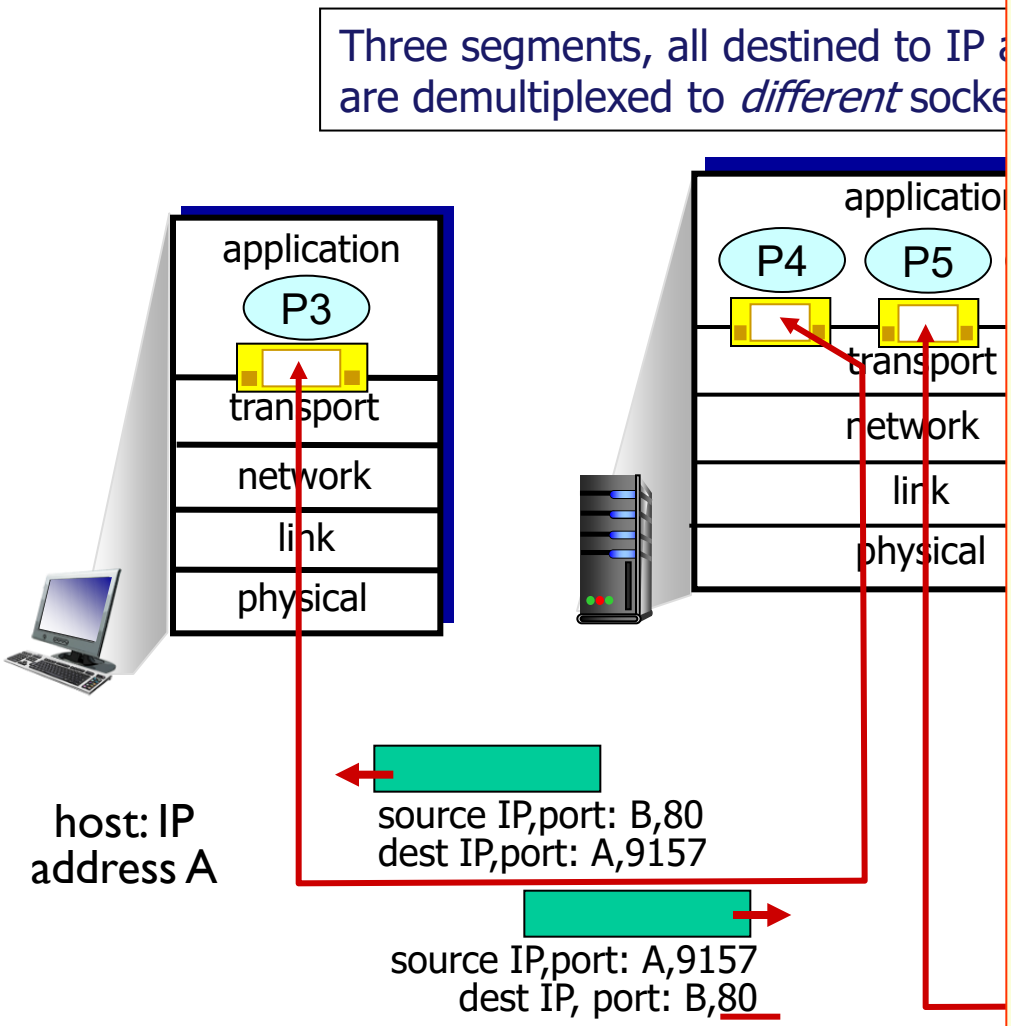


Connection-oriented (TCP)

Three segments, all destined to IP address: **B**, dest port: **80** are demultiplexed to *different* sockets



Connection-oriented (TCP)



- TCP socket identified by 4-tuple:
  - source IP address
  - source port number
  - dest IP address
  - dest port number
- demux: receiver uses all four values to direct segment to appropriate socket

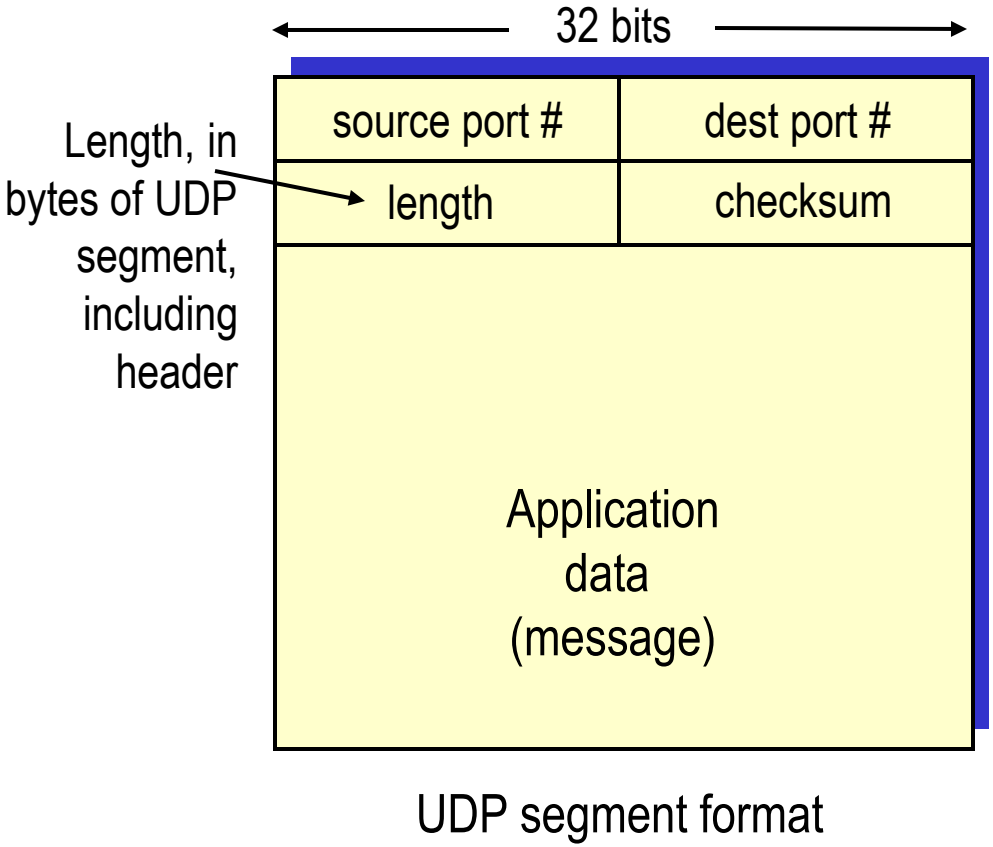
Today

- ✓ Consensus Algorithm – Byzantine failures
- ✓ Application level – APIs
- Transport layer
  - ✓ Recap
  - ✓ Multiplexing & Demultiplexing
  - UDP
  - TCP Error control + *in-order* byte-stream data transfer
    - What can go wrong



UDP: User Datagram Protocol [RFC 768]

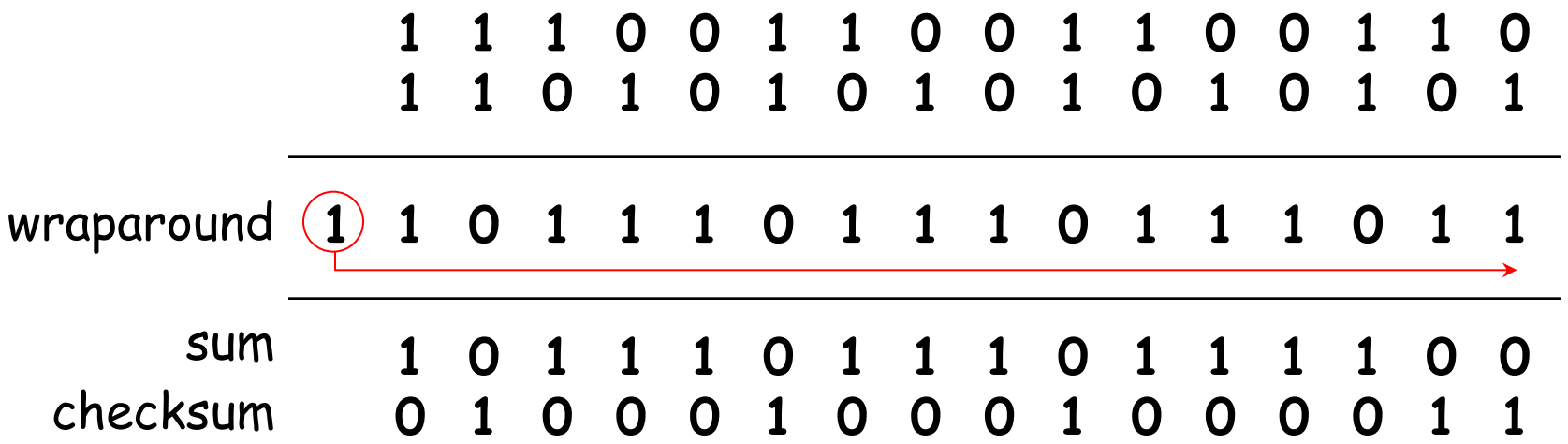
- Often used for streaming multimedia apps
  - loss tolerant; rate sensitive
- Other UDP uses
  - DNS; SNMP (Simple Network Management Protocol)
- Reliable transfer over UDP
  - add reliability at application layer or use **QUIC**
- Connectionless
  - no handshaking each segment is handled independently



UDP segment format

Internet Checksum Example

- Note
  - When adding numbers, a carryout from the most significant bit needs to be added to the result
- Example: add two 16-bit integers



UDP checksum

Goal: detect "errors" (e.g., flipped bits) in transmitte

Instead of comparing the checksums, the receiver adds its sum to the sender's checksum.
if the result is 1111 1111, the packet is considered correct, otherwise, damaged.

Sender:

- treat segment contents including header fields as sequence of 16-bit integers
- checksum: addition (1's complement sum) of segment contents
- sender puts checksum value into UDP checksum field

Receiver:

- compute checksum of received segment
- check if computed checksum equals checksum field value:
  - NO - error detected
  - YES - no error detected. *But maybe errors nonethless?*

UDP checksum: comments

- RFC1071 (http://www.faqs.org/rfcs/rfc1071.html) called “Computing the Internet Checksum” (17 pages) describes efficient implementations of checksum calculations and discuss why using 1’s complemet is useful.

- Damaged segments are either discarded or passed to the application with a warning (this depends on the specific implementation of UDP).

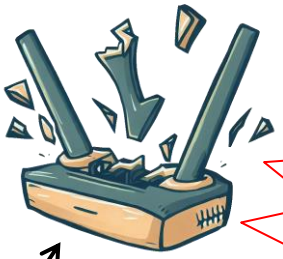
Today

- ✓ Consensus Algorithm – Byzantine failures
- ✓ Application level – APIs
- Transport layer
  - ✓ Recap
  - ✓ Multiplexing & Demultiplexing
  - ✓ UDP
  - TCP Error control + *in-order* byte-stream data transfer
    - What can go wrong

Principles of Reliable data transfer

The path of the packet is a **dangerous** one…





Malformed equipment



Getting Lost





Full queue



Bugs

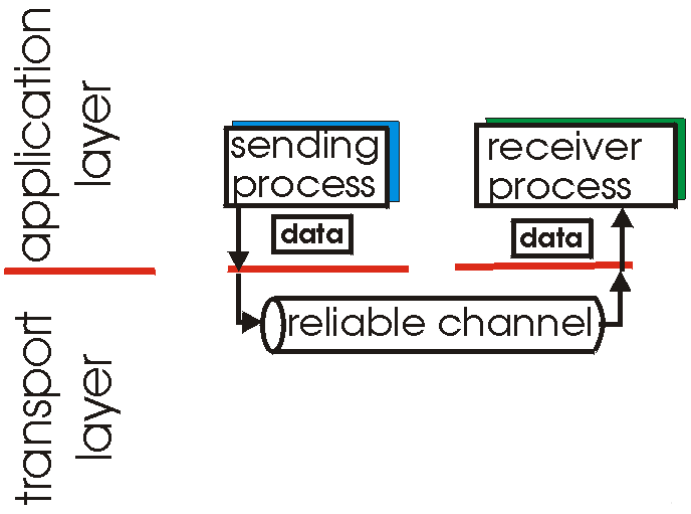


Radiation

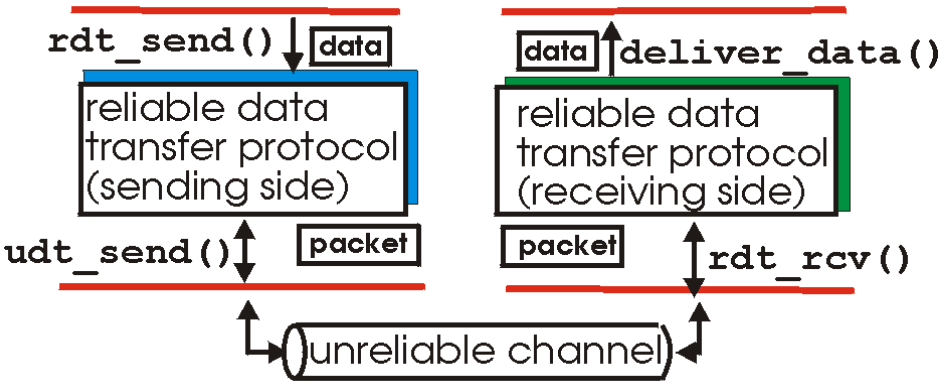
And more…

Principles of Reliable data transfer

Characteristics of unreliable channel will determine complexity of reliable data transfer protocol (rdt)

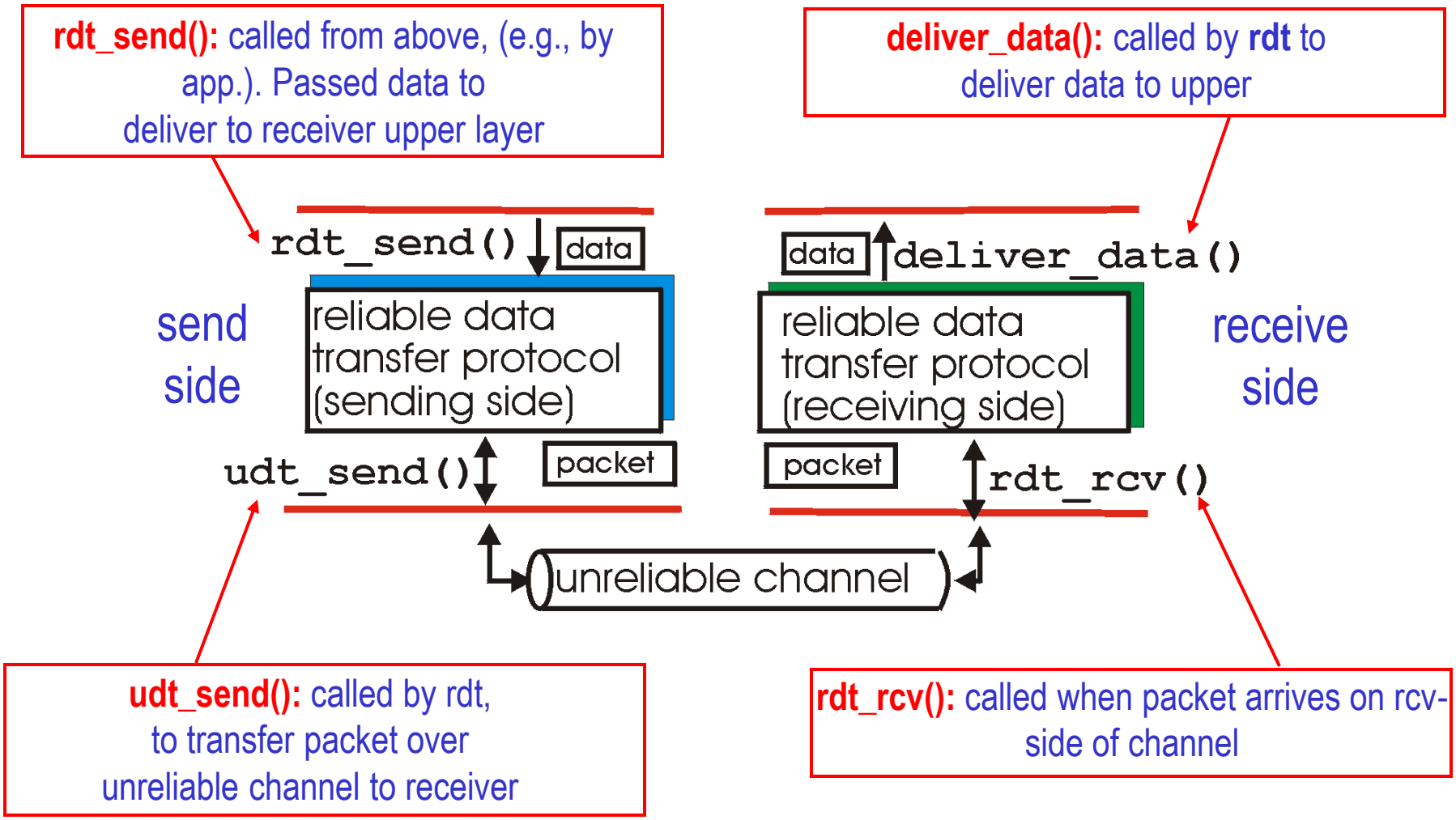


provided service



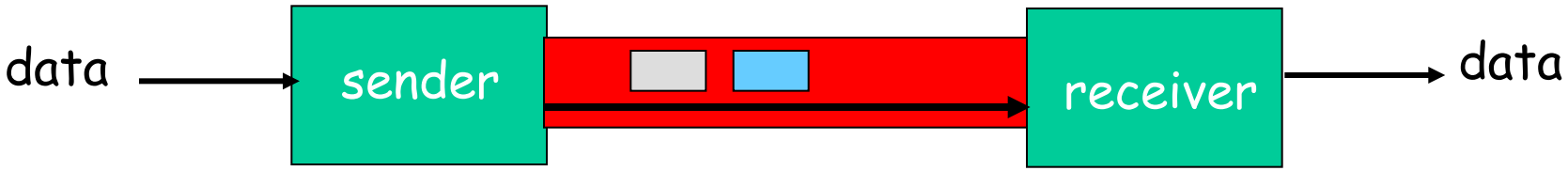
service implementation

Reliable data transfer: getting started



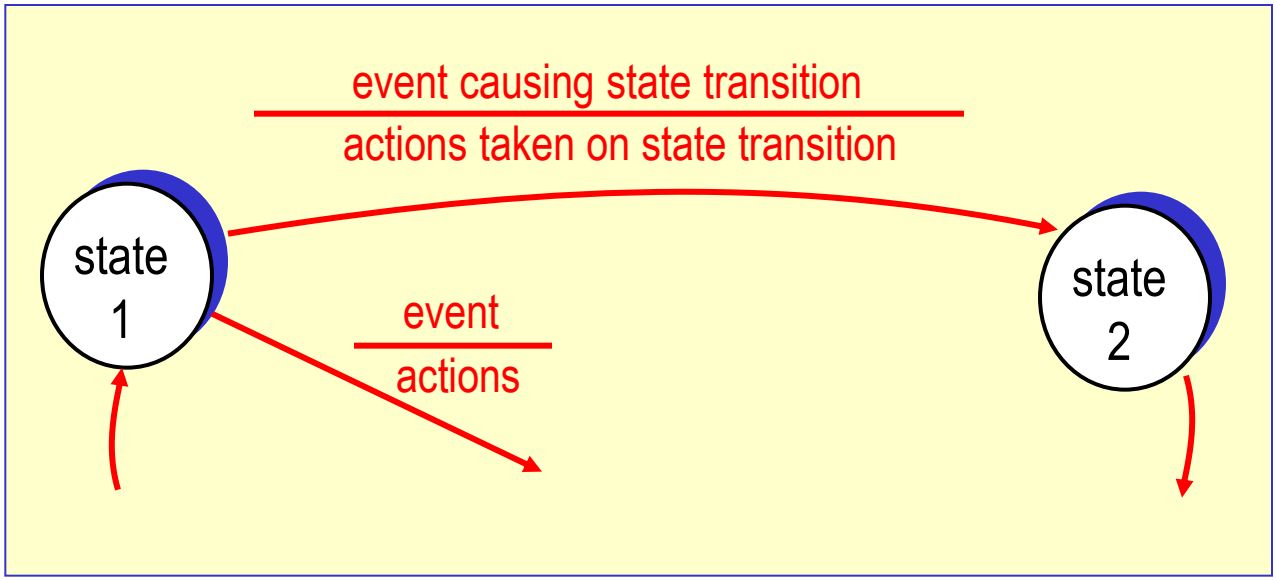
Unreliable Channel: Type of faults

1. Corrupted packets (bit errors)
2. Duplications
3. Loss
4. Reorder

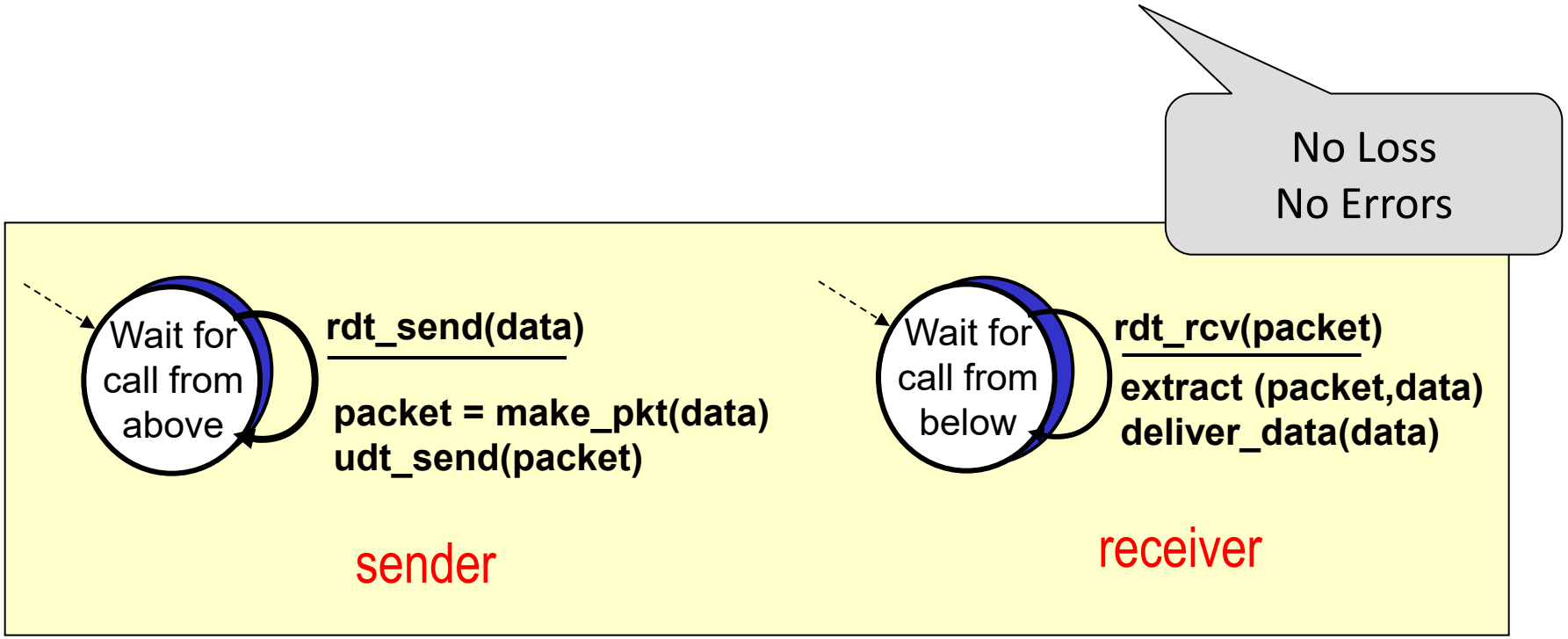


Reliable data transfer: getting started

We'll use finite state machines (FSM) to specify sender, receiver



Rdt1.0: reliable transfer over a reliable channel

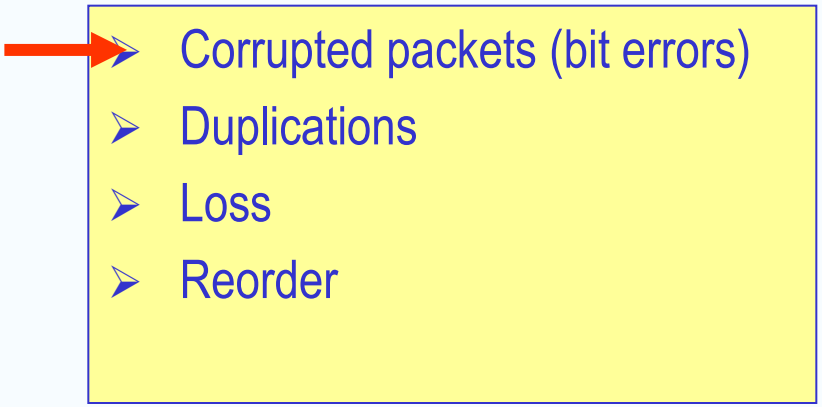


Today

- ✓ Consensus Algorithm – Byzantine failures
- ✓ Application level – APIs
- Transport layer
  - ✓ Recap
  - ✓ Multiplexing & Demultiplexing
  - ✓ UDP
  - TCP Error control + *in-order* byte-stream data transfer
    - ✓ What can go wrong
    - Error control
    - Duplication
    - Loss
    - Reorder

Rdt2.0: channel with bit errors

- Underlying channel may flip bits in packet
  - can use checksum to detect bit errors
- *The* question: how to recover from errors

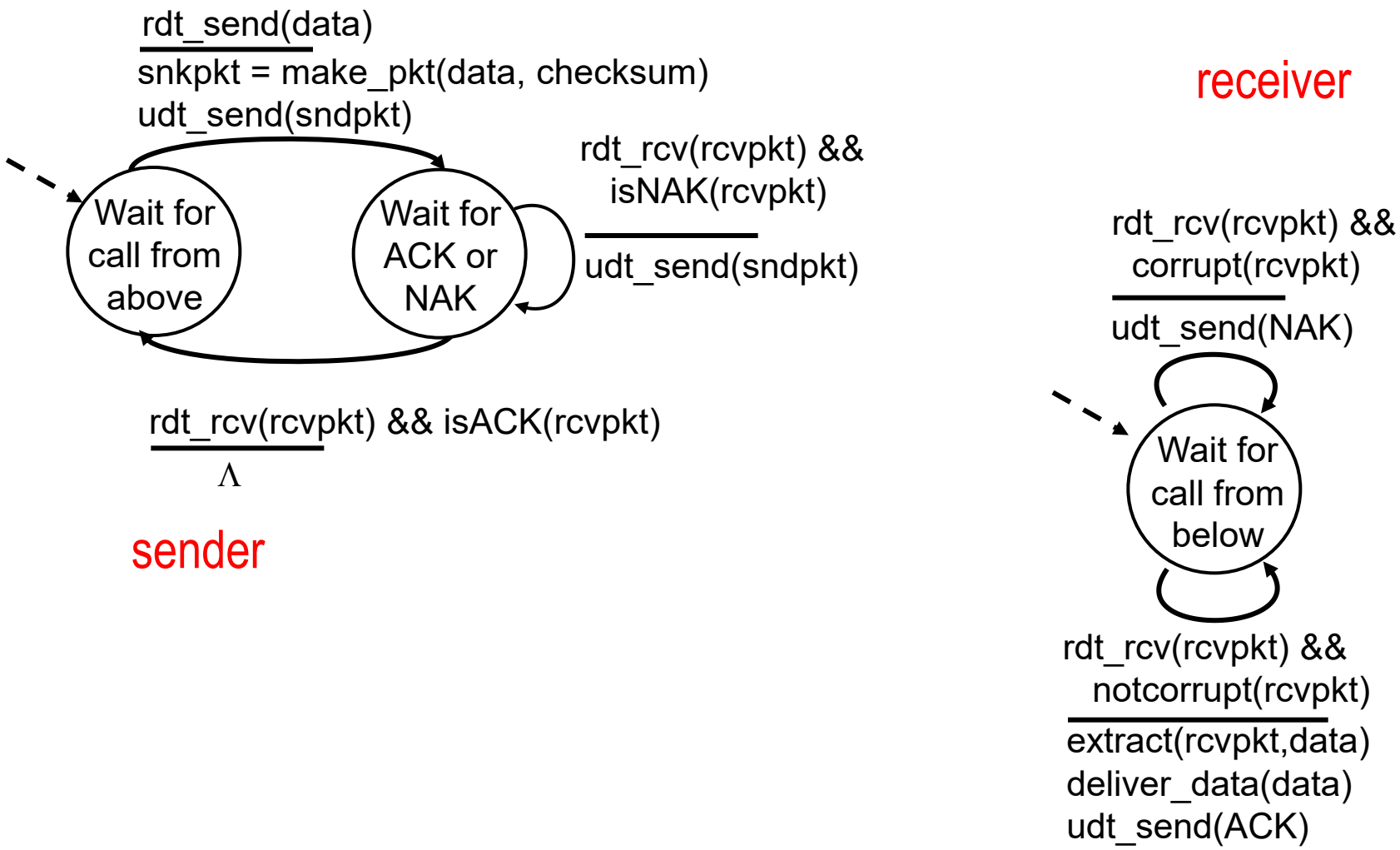


Rdt2.0: Channel with bit errors

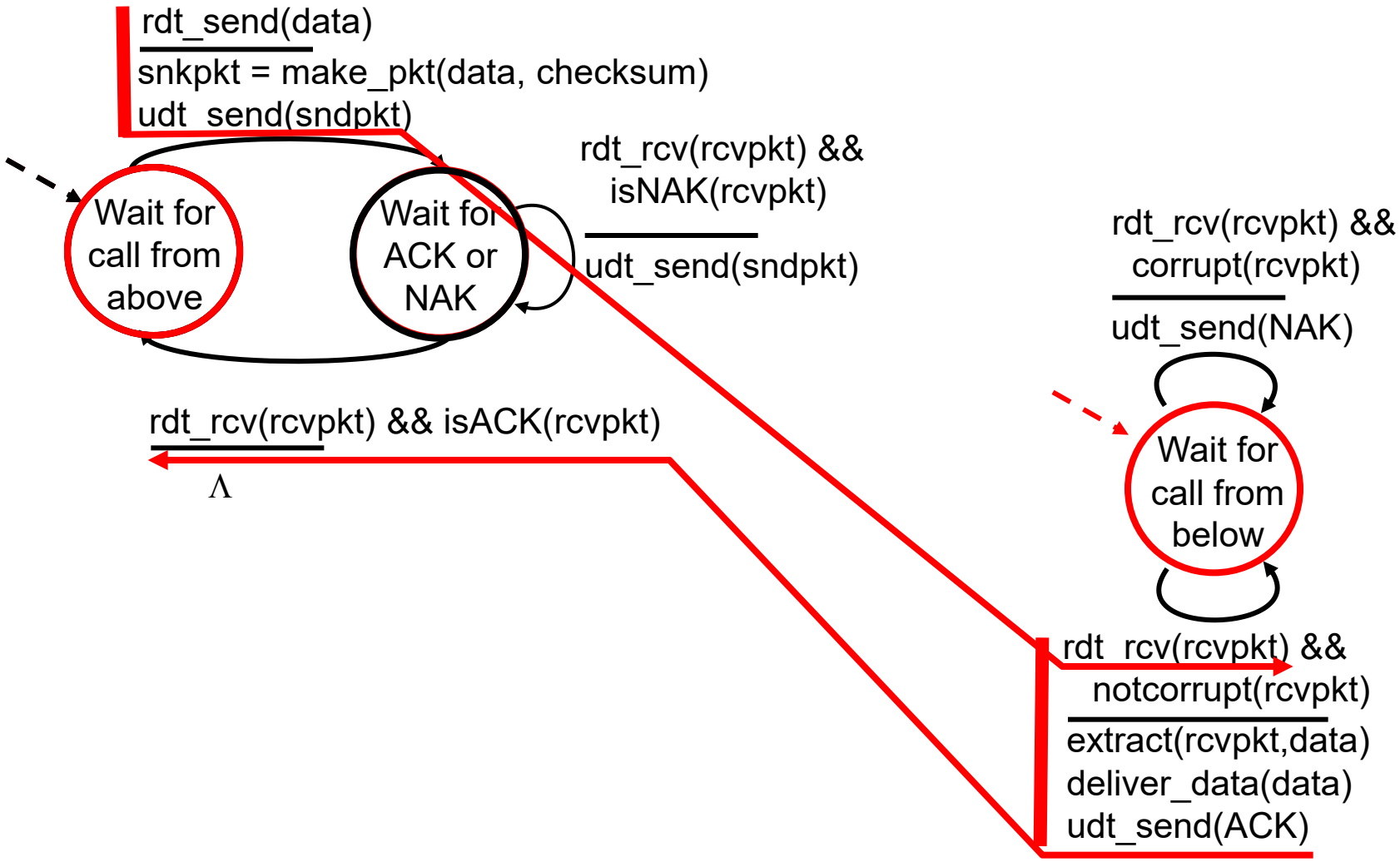
- Underlying channel may flip bits in packet
  - can use checksum to detect bit errors
- How to recover from errors?
  - *acknowledgements (ACKs):*
    receiver explicitly tells sender that pkt received OK
  - *negative acknowledgements (NAKs):*
    receiver explicitly tells sender that pkt had errors
  - sender retransmits pkt on receipt of NAK

stop and wait
Sender sends one packet,
then waits for receiver response

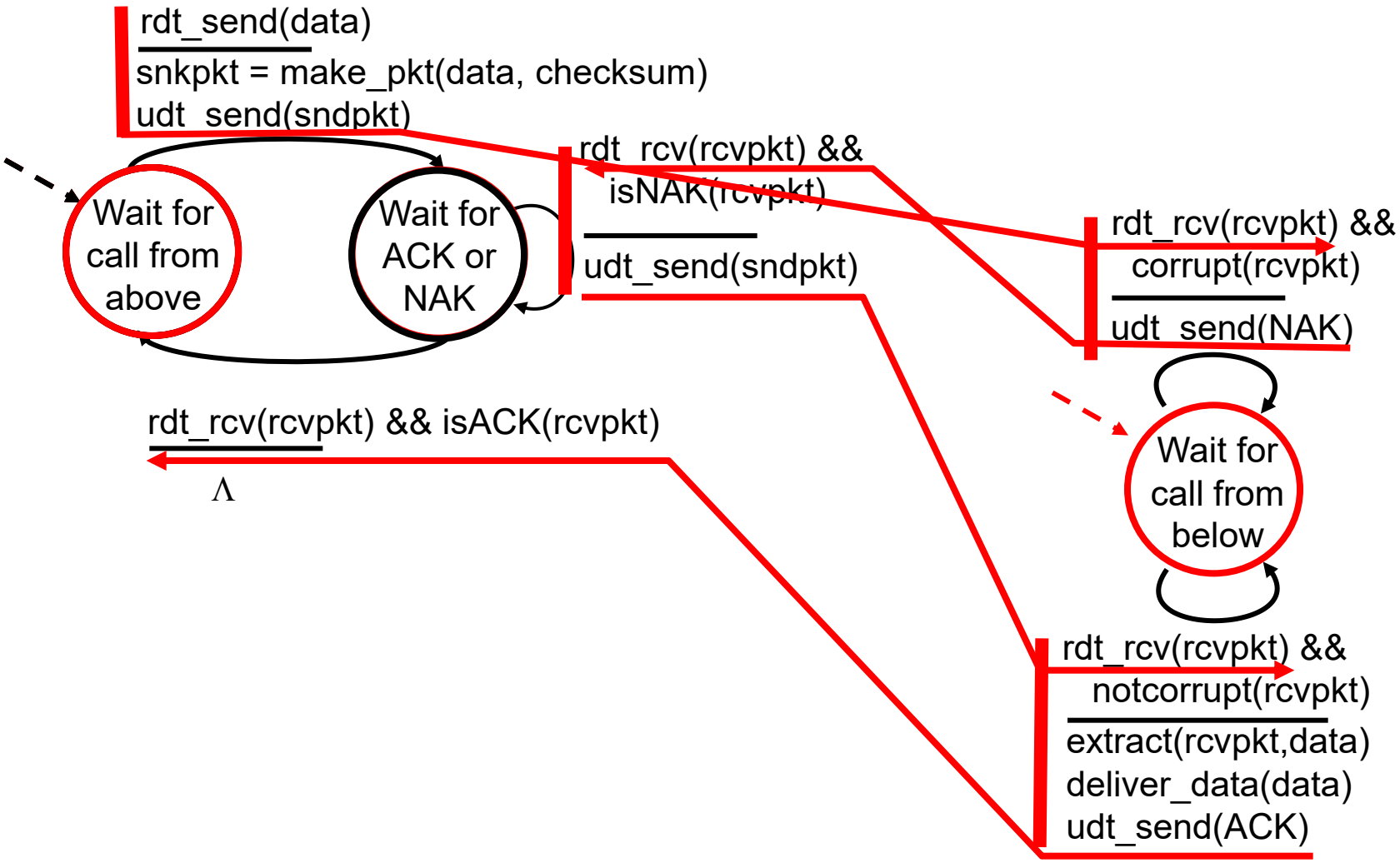
rdt2.0: FSM specification



rdt2.0: operation with no errors

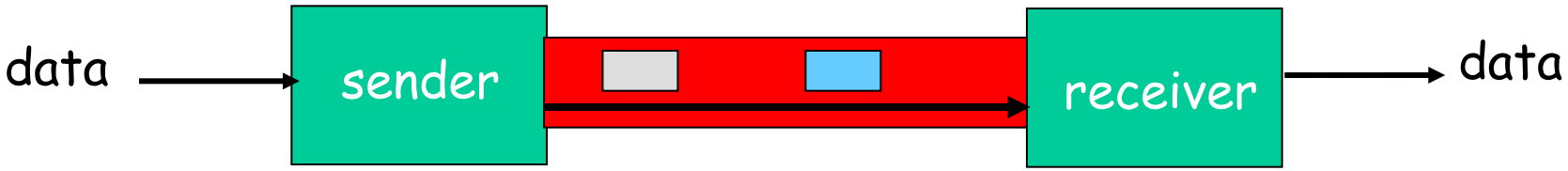


rdt2.0: error scenario



rdt2.0 has a fatal flaw!

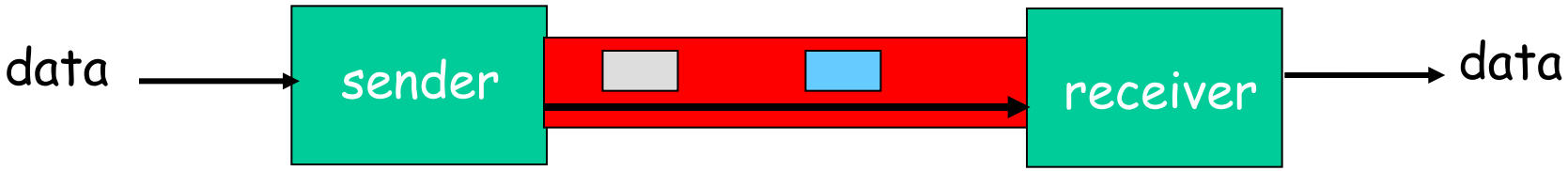
What happens if ACK/NAK corrupted? Assume sender always knows when corruption occurs.



rdt2.0 has a fatal flaw!

What happens if ACK/NAK corrupted? Assume sender always knows when corruption occurs.

- sender doesn't know what happened at receiver!
- Re-transmit



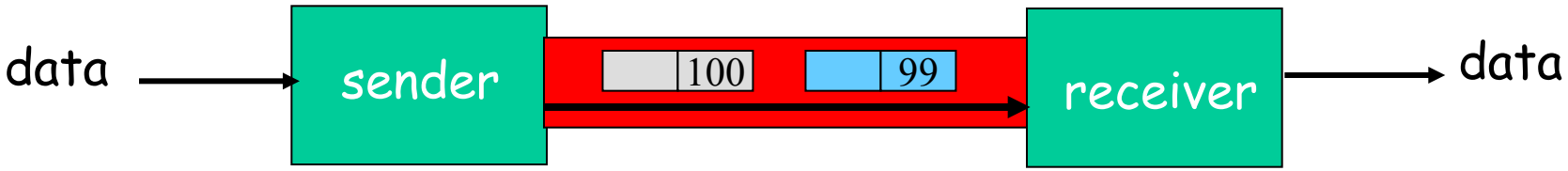
rdt2.0 has a fatal flaw!

What happens if ACK/NAK corrupted? Assume sender always knows when corruption occurs.

- sender doesn't know what happened at receiver!
- can't just retransmit -- possible duplicate

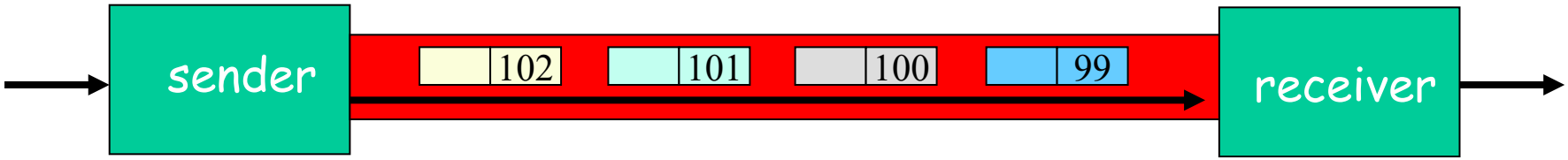
Handling duplicates:

- sender adds *sequence number* to each pkt
- sender retransmits current pkt if ACK/NAK garbled
- receiver discards (doesn't deliver up) duplicate pkt



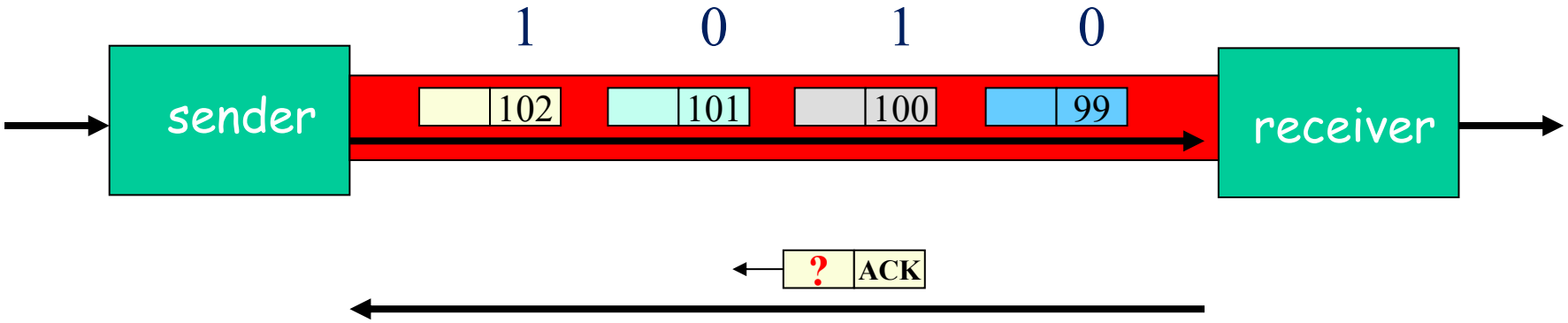
Question

How many different sequence numbers are needed ?

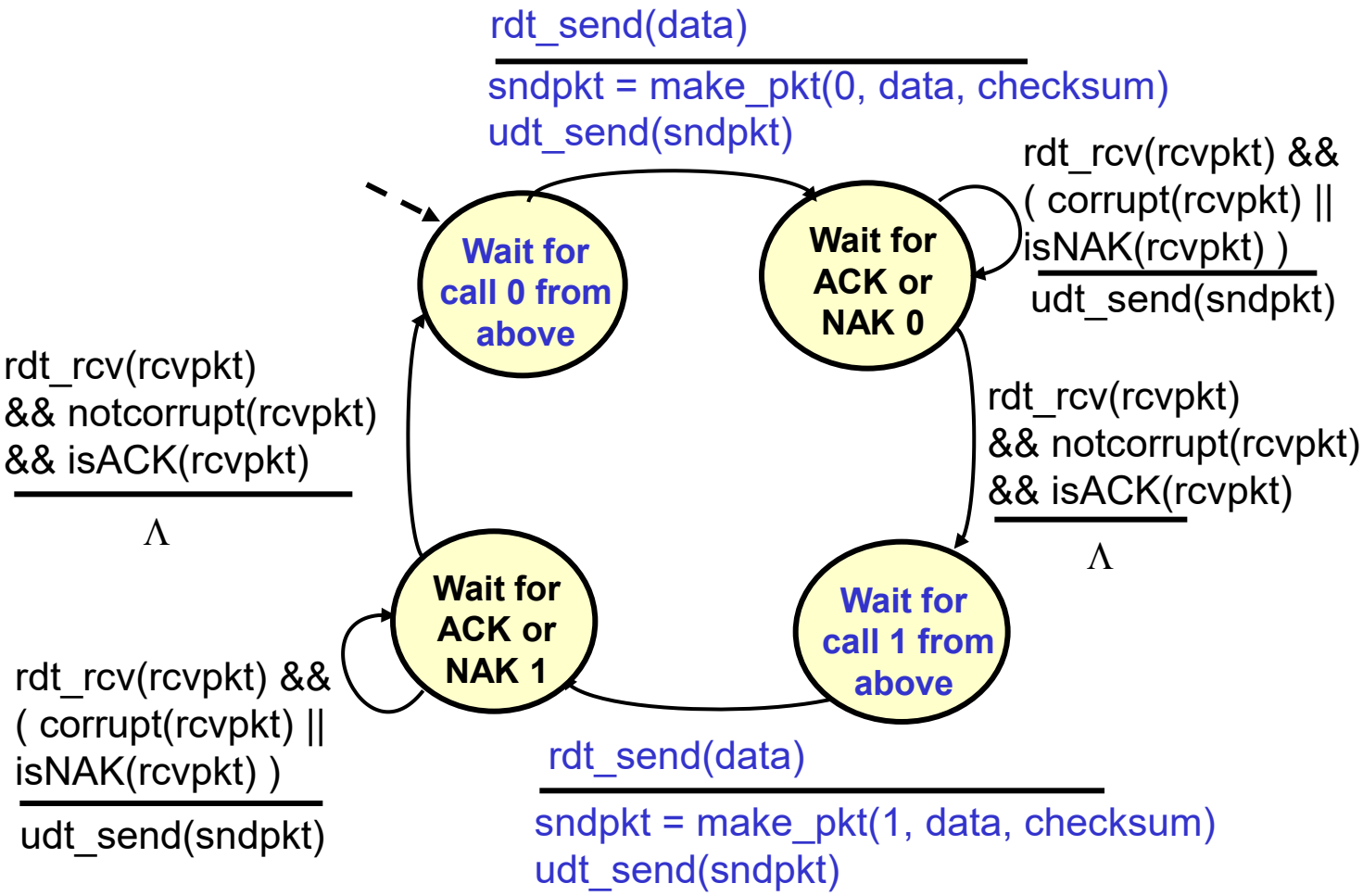


Question

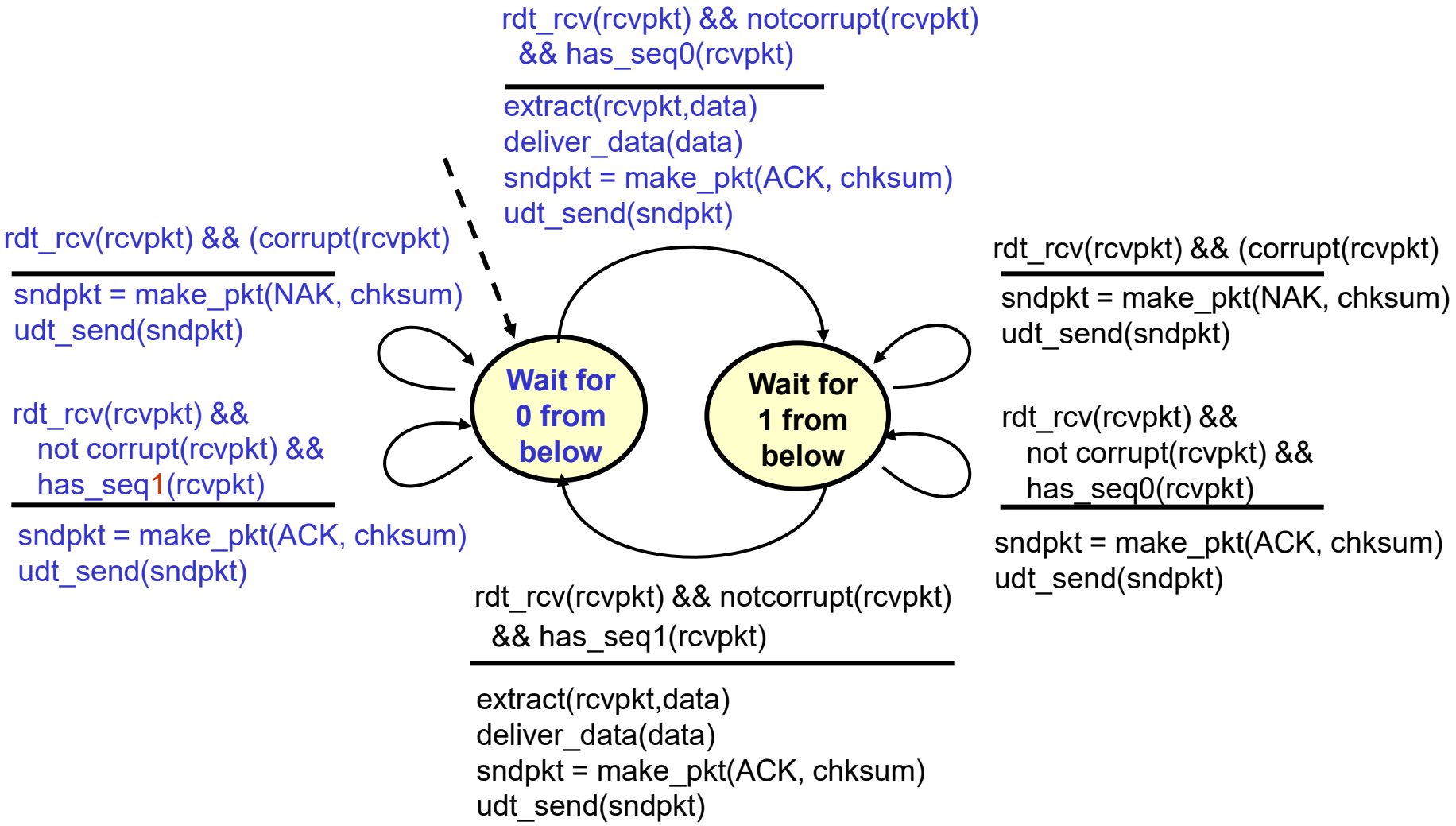
Do we need to add sequence numbers also for ACK/NAK ?



rdt2.1: sender, handles garbled ACK/NAKs

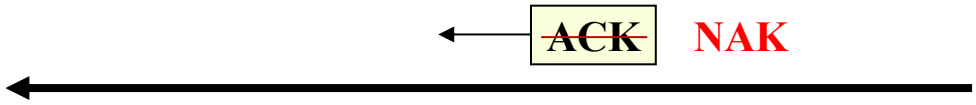


rdt2.1: receiver, handles garbled ACK/NAKs

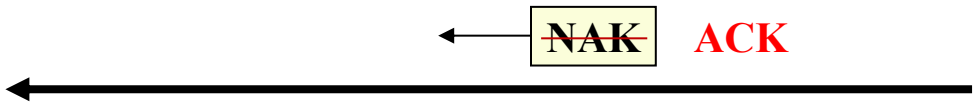


Questions

Would the algorithm work if ACK can be changed to NAK ?



Would the algorithm work if NAK can be changed to ACK ?



rdt2.1: discussion

Sender:

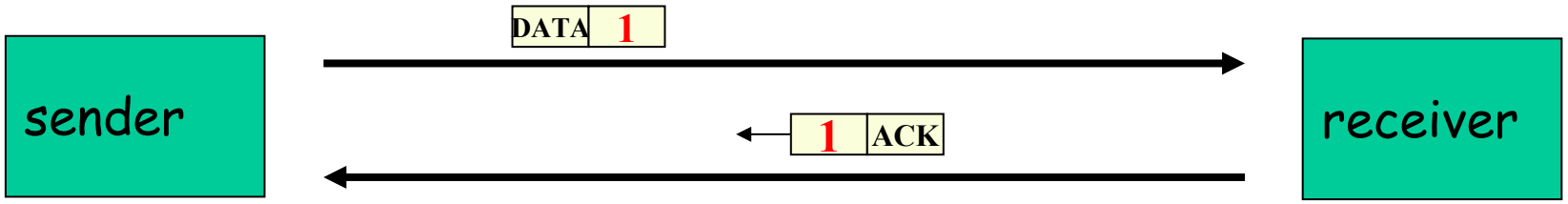
- seq # added to pkt
- two seq. #'s (0,1) will suffice. Why?
- must check if received ACK/NAK corrupted
- twice as many states

Receiver:

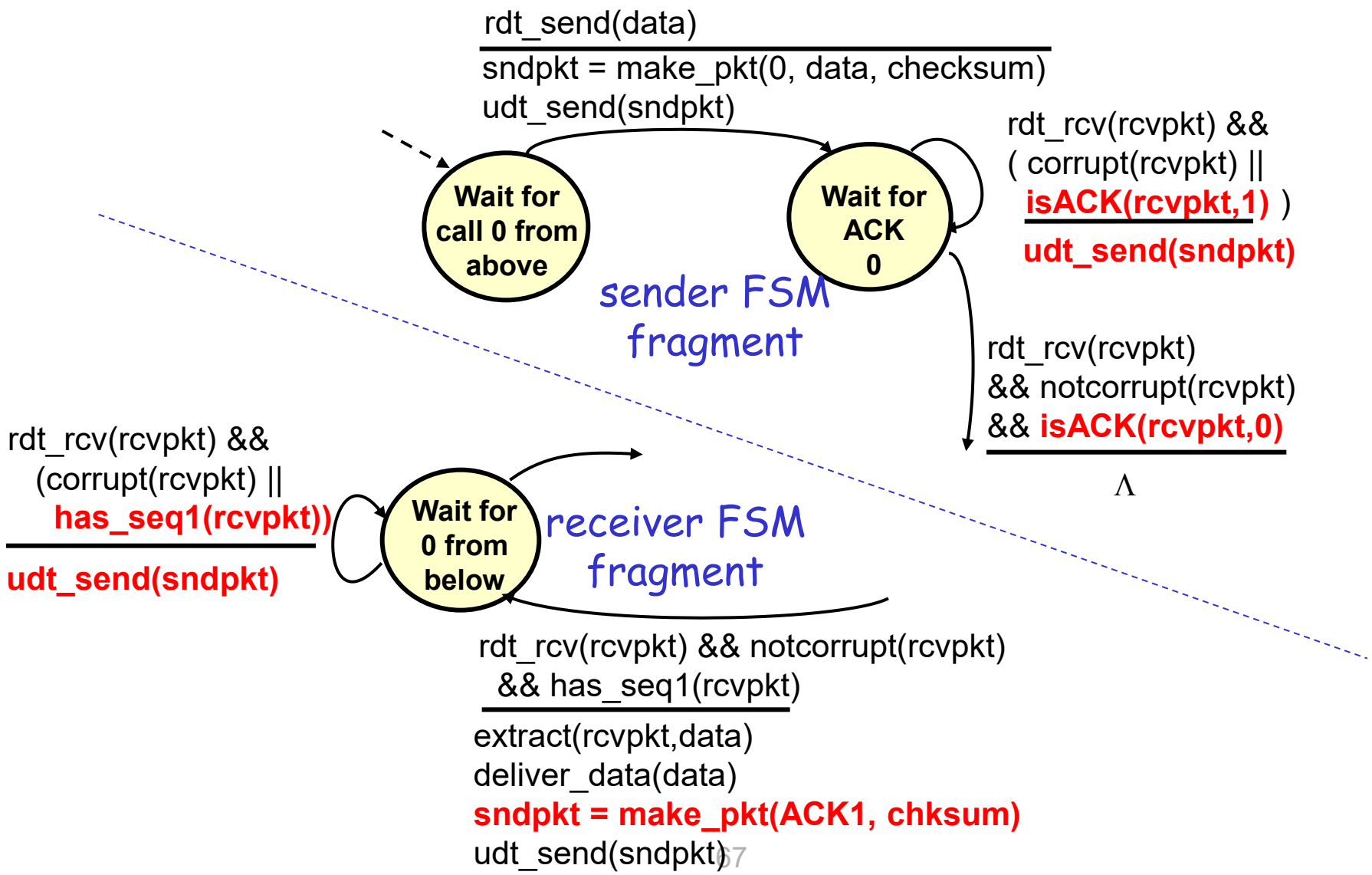
- must check if received packet is duplicate
  - state indicates whether 0 or 1 is expected pkt seq #
- note: receiver can *not* know if its last ACK/NAK received OK at sender

rdt2.2: a NAK-free protocol

- Same functionality as rdt2.1, using ACKs only
- Instead of NAK, receiver sends ACK for last pkt received OK
  - receiver must *explicitly* include seq # of pkt being ACKed
- Duplicate ACK at sender results in same action as NAK: *retransmit current pkt*



rdt2.2: sender, receiver fragments



Today

- ✓ Consensus Algorithm – Byzantine failures
- ✓ Application level – APIs
- Transport layer
  - ✓ Recap
  - ✓ Multiplexing & Demultiplexing
  - ✓ UDP
  - TCP Error control + *in-order* byte-stream data transfer
    - ✓ What can go wrong
    - ✓ Error control
    - Duplication
    - Loss
    - Reorder

Question

Would the algorithm work correctly assuming duplications ?

- Corrupted packets
- Duplications
- Loss
- Reorder

Today

- ✓ Consensus Algorithm – Byzantine failures
- ✓ Application level – APIs
- Transport layer
  - ✓ Recap
  - ✓ Multiplexing & Demultiplexing
  - ✓ UDP
  - TCP Error control + *in-order* byte-stream data transfer
    - ✓ What can go wrong
    - ✓ Error control
    - ✓ Duplication
    - Loss
    - Reorder

The alternating bit protocol

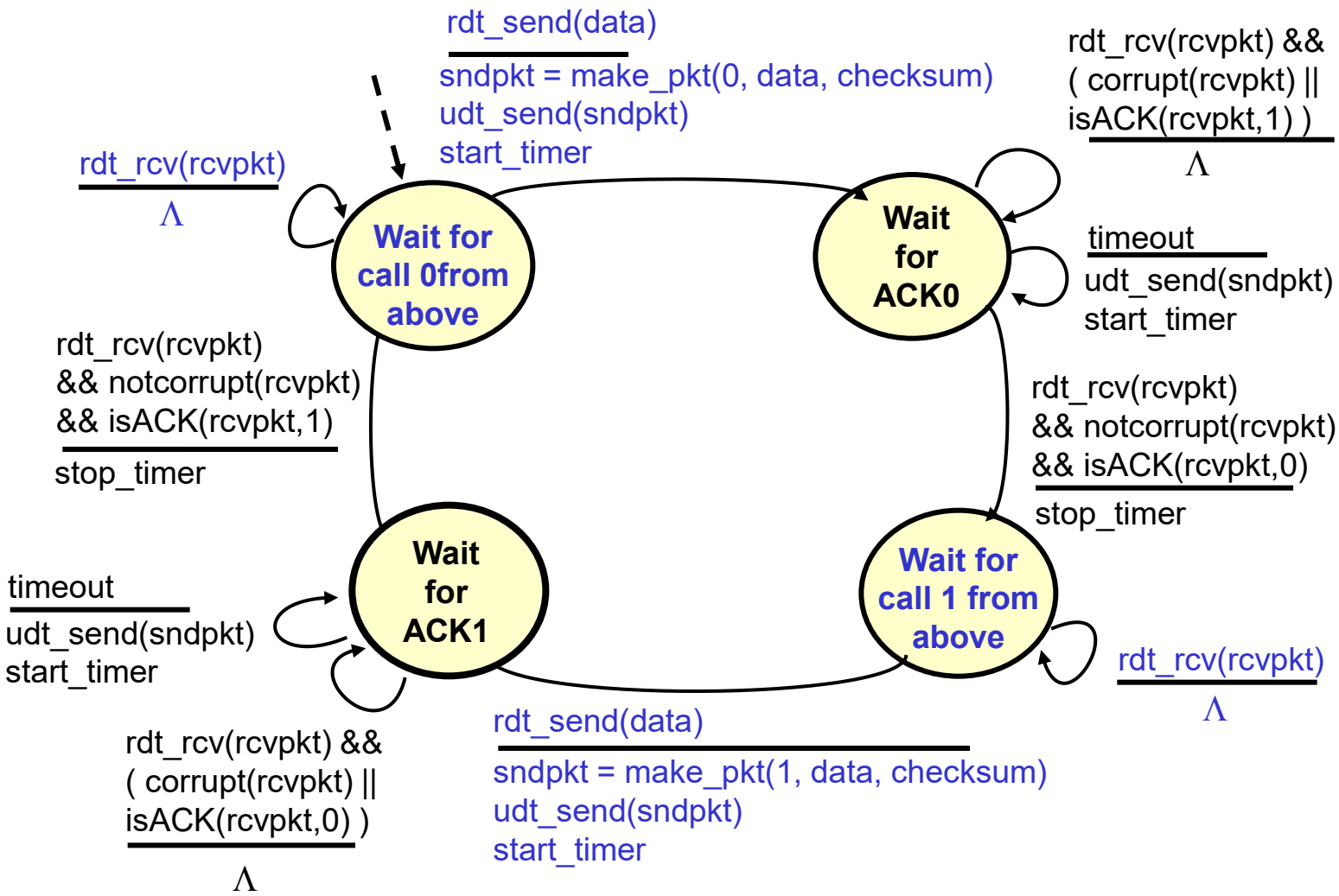
Approach: sender waits "reasonable" amount of time for ACK

- retransmits if no ACK received in this time
- if pkt (or ACK) just delayed (not lost):
  - retransmission can be duplicate, but use of seq. #'s already handles this
  - receiver must specify seq # of pkt being ACKed
- requires countdown timer

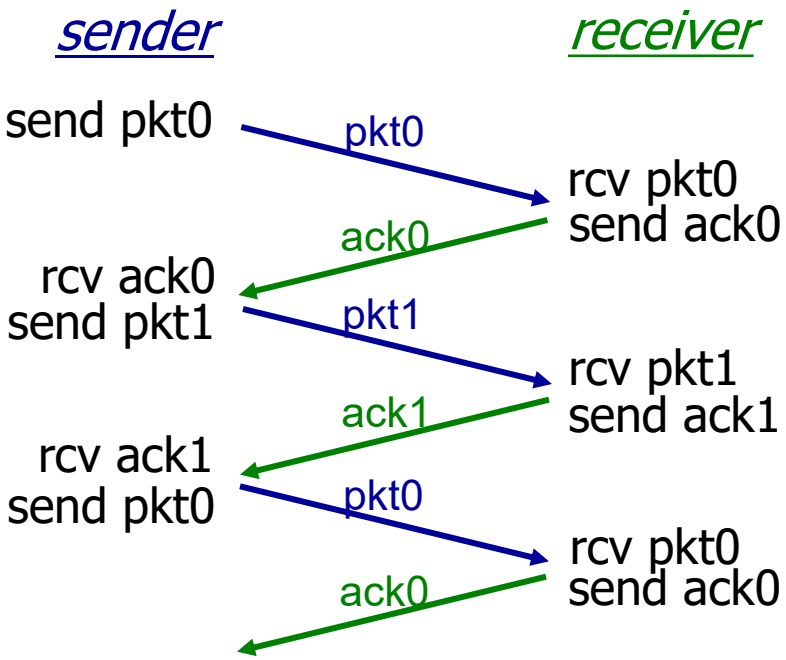
Question: A timer is required at:

1. sender only
2. receiver only
3. both sender & receiver

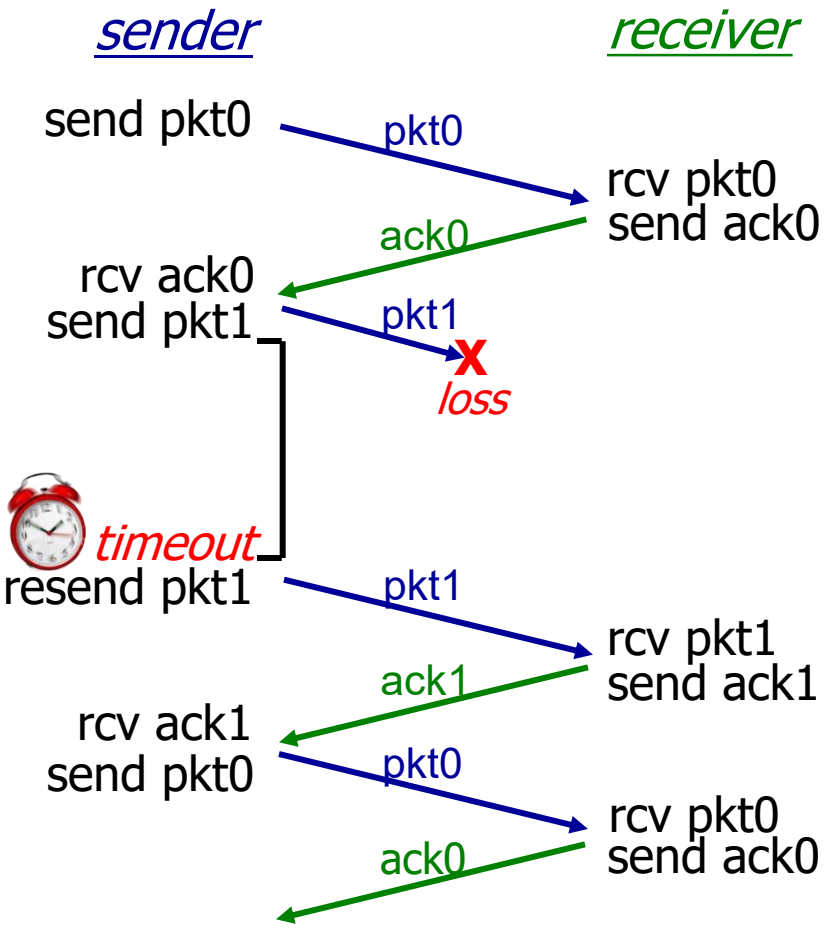
rdt3.0 sender



rdt3.0 in action

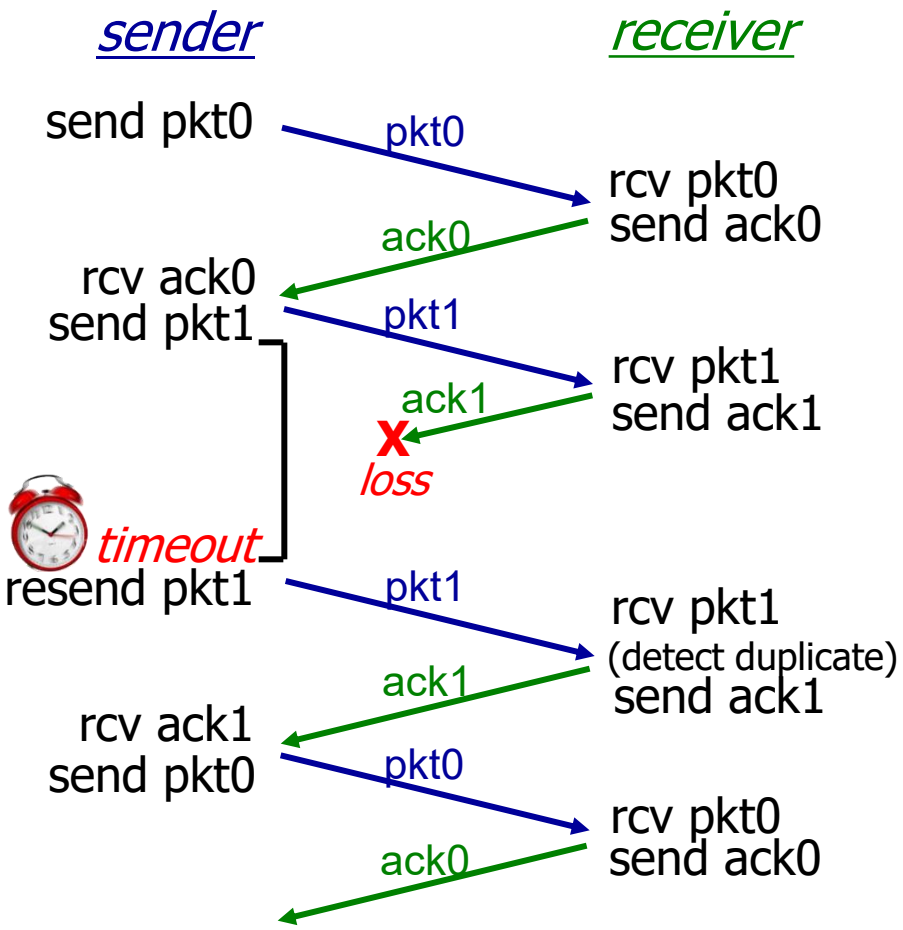


(a) no loss

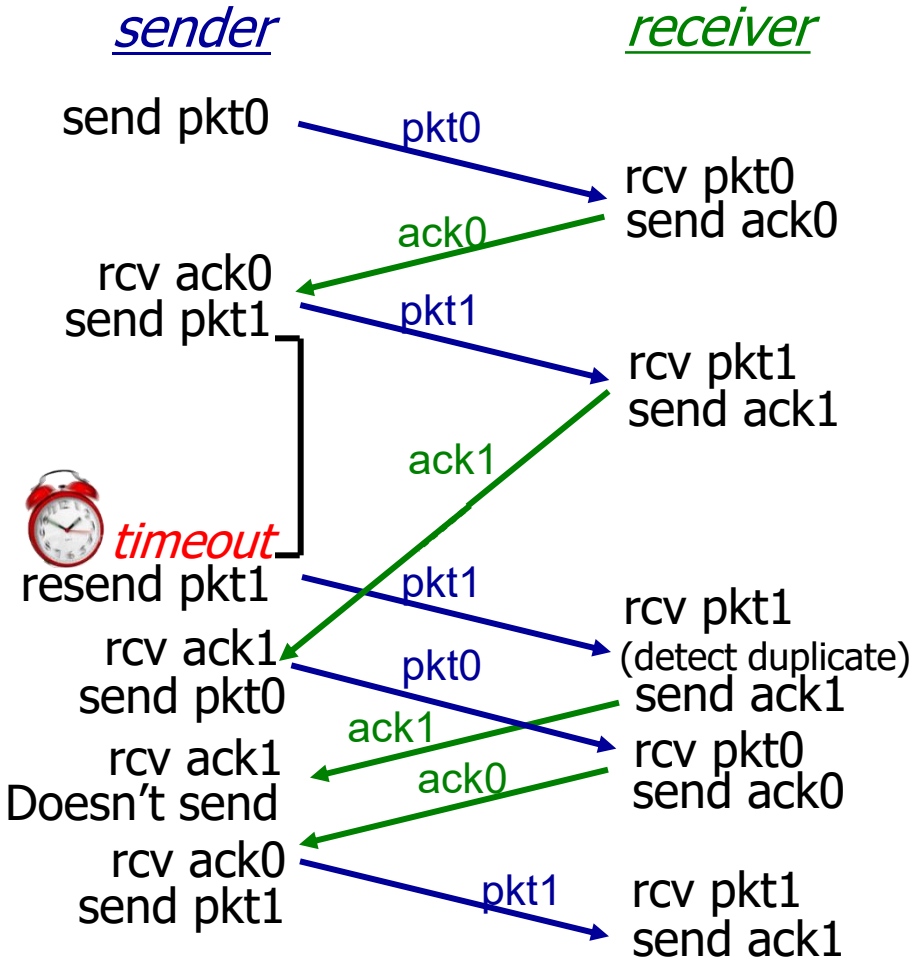


(b) packet loss

rdt3.0 in action



(c) ACK loss



(d) premature timeout/ delayed ACK

Today

- ✓ Consensus Algorithm – Byzantine failures
- ✓ Application level – APIs
- Transport layer
  - ✓ Recap
  - ✓ Multiplexing & Demultiplexing
  - ✓ UDP
  - TCP Error control + *in-order* byte-stream data transfer
    - ✓ What can go wrong
    - ✓ Error control
    - ✓ Duplication
    - ✓ Loss
    - Reorder